



Práctica de laboratorio 4

1. Ruido cuántico

(a) (6 pts) Contesta las siguientes preguntas:

i. Demuestra que para las matrices de Pauli se cumple que $\hat{\sigma}_j \hat{\sigma}_k = \delta_{jk} \hat{I} + i \sum_{l=1}^3 \epsilon_{jkl} \hat{\sigma}_l$.

ii. Demostrar que $\frac{I}{2} = \frac{\rho + X\rho X + Y\rho Y + Z\rho Z}{4}$ y con eso explicar los canales de Kraus del Depolarizing Channel.

Solución.

P.d. Para las matrices de Pauli se cumple: $\hat{\sigma}_j \hat{\sigma}_k = \delta_{jk} \hat{I} + i \sum_{l=1}^3 \epsilon_{jkl} \hat{\sigma}_l$

Consideremos: $\hat{\sigma}_1 = X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, $\hat{\sigma}_2 = Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$, $\hat{\sigma}_3 = Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$.

Y como recuerdo:

- δ_{jk} (delta de Kronecker), definida como: $\delta_{jk} = \begin{cases} 1, & j = k, \\ 0, & j \neq k, \end{cases}$
- ϵ_{jkl} (Levi-Civita), definido como: $\epsilon_{jkl} = \begin{cases} 1, & (j, k, l) \text{ es una permutación par de } (1, 2, 3) \\ -1, & (j, k, l) \text{ es una permutación impar de } (1, 2, 3) \\ 0, & \text{si dos índices son iguales} \end{cases}$

Lo interesante sucede cuando vemos que δ_{jk} y ϵ_{jkl} dependen directamente de si los índices son iguales o distintos. En particular, cuando $j = k$, se tiene $\delta_{jk} = 1$ y $\epsilon_{jkl} = 0$, mientras que para $j \neq k$, ocurre lo contrario. Por ello, podemos proceder con la demostración en los casos $j = k$ y $j \neq k$.

- Consideremos el caso $j = k$

$$\begin{aligned} X^2 &= \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I \\ Y^2 &= \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = \begin{pmatrix} (-i)i & 0 \\ 0 & i(-i) \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I \\ Z^2 &= \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I \end{aligned}$$

Por lo tanto, para cualquier $j \in \{1, 2, 3\}$, $\hat{\sigma}_j^2 = I$.

Ahora evaluamos el lado derecho de la identidad. Como $j = k$, entonces $\delta_{jk} = 1$, y además $\epsilon_{jkl} = 0$ para todo l , ya que el símbolo de Levi-Civita se anula cuando dos índices son iguales. Por tanto,

$$\delta_{jk} I + i \sum_{l=1}^3 \epsilon_{jkl} \hat{\sigma}_l = I + i \sum_{l=1}^3 0 \cdot \hat{\sigma}_l = I$$

Entonces, para el caso $j = k$, ambos lados de la igualdad dan I , y por lo tanto la identidad se cumple.

- Consideremos el caso $j \neq k$

En este caso, por definición de la delta de Kronecker, $\delta_{jk} = 0$. Así, la identidad que queremos demostrar se reduce a:

$$\hat{\sigma}_j \hat{\sigma}_k = i \sum_{l=1}^3 \epsilon_{jkl} \hat{\sigma}_l$$

I. e. hay que Over que el producto de dos matrices de Pauli distintas produce la tercera matriz de Pauli, multiplicada por un factor i y con un signo determinado por el símbolo de Levi-Civita.

$$XY = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = \begin{pmatrix} i & 0 \\ 0 & -i \end{pmatrix} = iZ$$

$$YZ = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} 0 & i \\ i & 0 \end{pmatrix} = iX$$

$$ZX = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} = iY$$

$$\hat{\sigma}_1 \hat{\sigma}_2 = i\hat{\sigma}_3, \quad \hat{\sigma}_2 \hat{\sigma}_3 = i\hat{\sigma}_1, \quad \hat{\sigma}_3 \hat{\sigma}_1 = i\hat{\sigma}_2$$

Estos tres productos corresponden a las permutaciones pares de $(1, 2, 3)$, por lo que $\epsilon_{123} = \epsilon_{231} = \epsilon_{312} = 1$.

Así, si (j, k, l) es una permutación par, entonces: $\hat{\sigma}_j \hat{\sigma}_k = i\hat{\sigma}_m = i\epsilon_{jkm} \hat{\sigma}_m$

Como m es el único índice distinto de j y k , en la suma $\sum_{l=1}^3 \epsilon_{jkl} \hat{\sigma}_l$ todos los términos se anulan excepto el correspondiente a $l = m$. Por tanto, $\sum_{l=1}^3 \epsilon_{jkl} \hat{\sigma}_l = \epsilon_{jkm} \hat{\sigma}_m$ y entonces $\hat{\sigma}_j \hat{\sigma}_k = i \sum_{l=1}^3 \epsilon_{jkl} \hat{\sigma}_l$.

Además, al invertir el orden de dos matrices de Pauli distintas, el signo cambia:

$$YX = -iZ, ZY = -iX, XZ = -iY$$

En notación de índices, $\hat{\sigma}_2 \hat{\sigma}_1 = -i\hat{\sigma}_3, \hat{\sigma}_3 \hat{\sigma}_2 = -i\hat{\sigma}_1, \hat{\sigma}_1 \hat{\sigma}_3 = -i\hat{\sigma}_2$.

Estos tres productos corresponden a las permutaciones impares de $(1, 2, 3)$, por lo que $\epsilon_{213} = \epsilon_{321} = \epsilon_{132} = -1$.

Así, si (j, k, l) es una permutación impar, entonces: $\hat{\sigma}_j \hat{\sigma}_k = -i\hat{\sigma}_m = i\epsilon_{jkm} \hat{\sigma}_m$.

Nuevamente, como m es el único índice distinto de j y k , $\sum_{l=1}^3 \epsilon_{jkl} \hat{\sigma}_l = \epsilon_{jkm} \hat{\sigma}_m$, por lo que $\hat{\sigma}_j \hat{\sigma}_k = i \sum_{l=1}^3 \epsilon_{jkl} \hat{\sigma}_l$.

Así, la identidad queda demostrada para el caso $j \neq k$.

■

(b) (6 pts)

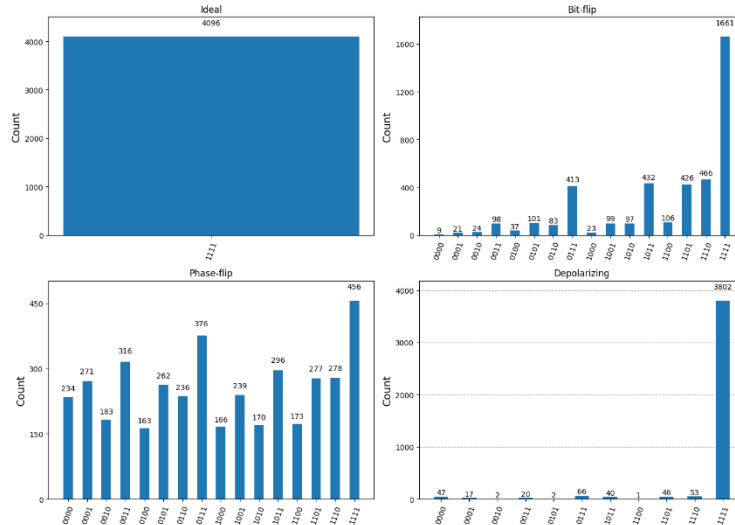
Corre Bernstein-Vazirani para el string que tú desees. Pero implementa los siguientes modelos de ruido:

- Bit-Flip con $p = 0.2$ en las compuertas H .
- Phase flip en todas las compuertas unitarias.
- Depolarizing channel con probabilidad $p = 0.01$ en todas las compuertas.

Explica los resultados que viste en el sistema.

Solución.

El código viene adjuntado al final del documento.



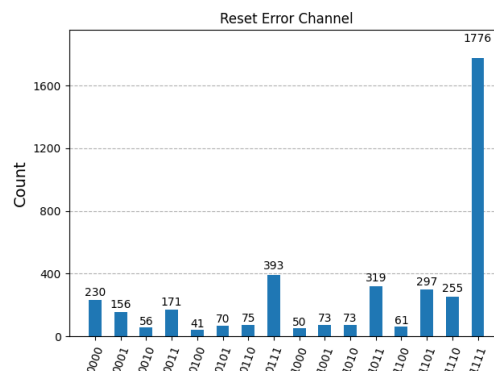
(c) (6 pts)

¿Existe algún canal de ruido que no se encuentre entre los primeros que mencionamos en la sesión? Investígalo, explícalo e impleméntalo utilizando los operadores de Kraus que correspondan a dicho canal y explica los resultados.

Solución.

Sí, en este caso decidí el Reset Error Channel, que por lo que comprendí es un modelo de ruido en el que un qubit puede perder completamente su estado cuántico y ser reinicializado a $|0\rangle$ o $|1\rangle$ con cierta probabilidad. Esto no solo altera una propiedad específica del qubit (como los anteriores), sino que puede borrar directamente la información almacenada en él, afectando significativamente el comportamiento del algoritmo cuántico.

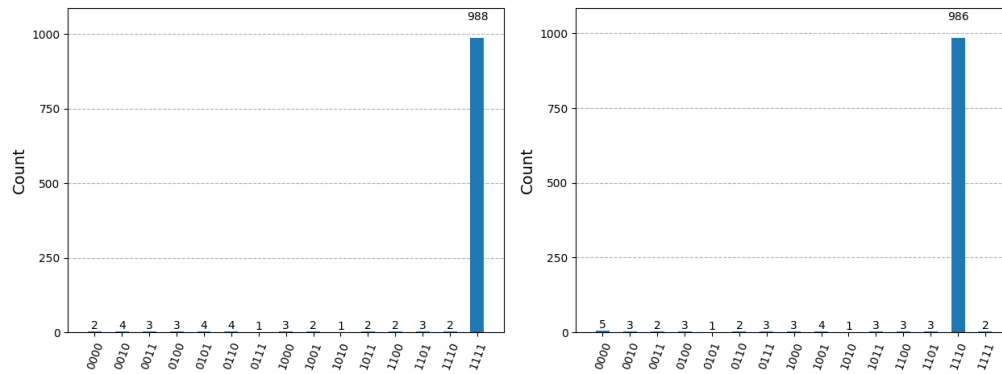
El código viene adjuntado al final del documento.



2. Grover

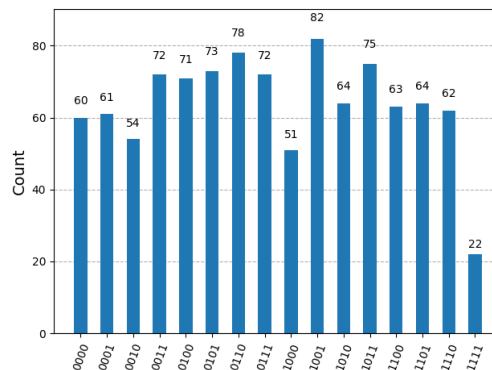
(a) (6 pts) Aplica el algoritmo de Grover para $|1111\rangle$ y para $|1110\rangle$ sin utilizar las funciones dadas para el oráculo de Grover y para el difusor de Grover; en su lugar, explica desde cero cómo realizaste el cambio de base para poder simular la operación del oráculo y del difusor. Realiza las iteraciones necesarias.

El código viene adjuntado al final del documento.



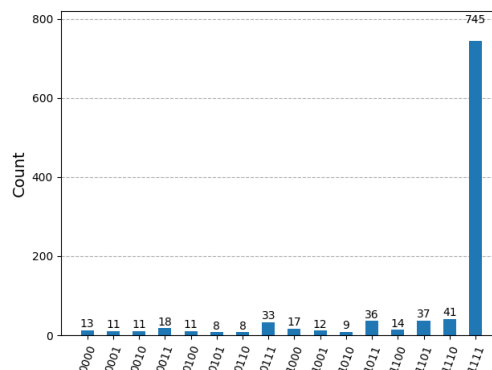
(b) (6 pts) Corre el algoritmo de Grover para encontrar el número de tu preferencia, pero aplica el operador de Grover una cantidad mayor de veces que la predicha por la teoría mencionada en el Notebook. ¿Qué pasa? ¿Hay algún cambio en las probabilidades? Explica por qué ocurre el cambio que observas en las probabilidades.

El código viene adjuntado al final del documento.



(c) (6 pts) Implementa algún modelo de ruido cuántico en las compuertas de 1 qubit del algoritmo de Grover, con probabilidad $p = 0.01$, y visualiza cómo cambia la implementación del algoritmo y si ahora funciona correctamente. Haz las comparaciones utilizando histogramas y explica por qué crees que mediste esos resultados.

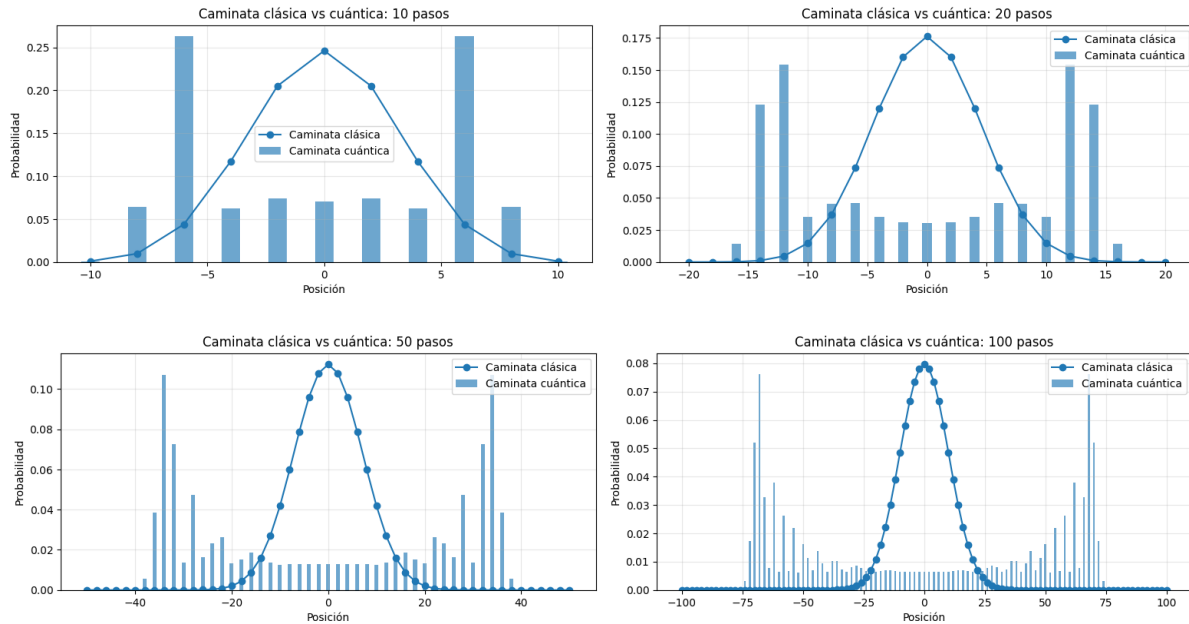
El código viene adjuntado al final del documento.



3. Caminatas Cuánticas

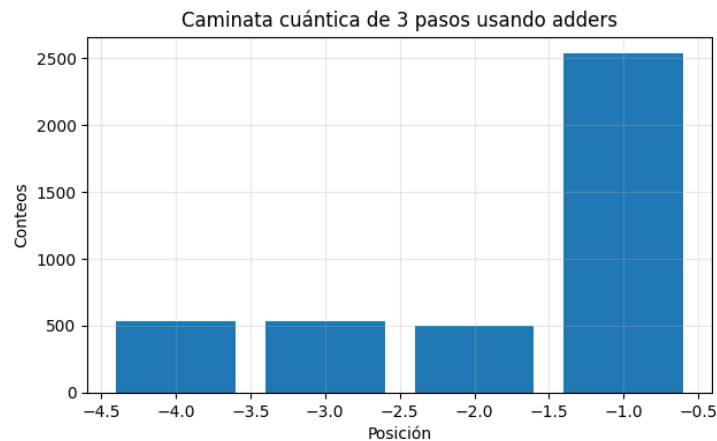
(a) (6 pts) Simula caminatas cuánticas para 10, 20, 50 y 100 pasos, y compáralas con caminatas clásicas similares con la misma cantidad de pasos. Explica cómo se visualizan, si son parecidas o diferentes. Utiliza gráficas o histogramas que ayuden a entender la intuición.

El código viene adjuntado al final del documento.



(b) (6 pts) Digamos que quisiera correr una caminata cuántica en hardware real, de modo que quisiera hacer una caminata de unos 3 pasos. ¿Es eso posible? Investiga cómo implementarla con *adders* y simula una caminata de 3 pasos en Qiskit. Tienes permiso de utilizar IA para este ejercicio, pero debes explicar con detalle cada cosa que la IA genere y realizar citas a fuentes reales que fundamenten lo que estás realizando. No dejes ningún cabo suelto.

El código viene adjuntado al final del documento.



4. QFT y QPE

(a) (6 pts) Demuestra a partir de la siguiente fórmula si la QFT es unitaria:

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{\frac{2\pi i x k}{2^n}} |k\rangle$$

Solución.

Partamos de la def. de la QFT sobre n qubits, considerando $N = 2^n$:

$$F|x\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{\frac{2\pi i x k}{N}} |k\rangle$$

P. d. la QFT es unitaria, i. e. $F^\dagger F = I$.

Sabemos que $F^\dagger F = I \Leftrightarrow$ los estados $F|x\rangle$ forman una base ortonormal t. q. $\langle Fx|Fy\rangle = \langle x|y\rangle = \delta_{xy}$.

Tomemos dos estados de la base computacional ($|x\rangle$ y $|y\rangle$) y calculamos el producto interno entre ambos estados transformados:

$$\langle Fy|Fx\rangle = \left(\frac{1}{\sqrt{N}} \sum_{l=0}^{N-1} e^{-\frac{2\pi i y l}{N}} \langle l| \right) \left(\frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{\frac{2\pi i x k}{N}} |k\rangle \right) = \frac{1}{N} \sum_{l=0}^{N-1} \sum_{k=0}^{N-1} e^{-\frac{2\pi i y l}{N}} e^{\frac{2\pi i x k}{N}} \langle l|k\rangle$$

Como la base computacional es ortonormal, $\langle l|k\rangle = \delta_{lk} \Rightarrow$ solo sobreviven los términos donde $l = k$:

$$\langle Fy|Fx\rangle = \frac{1}{N} \sum_{k=0}^{N-1} e^{-\frac{2\pi i y k}{N}} e^{\frac{2\pi i x k}{N}} = \frac{1}{N} \sum_{k=0}^{N-1} e^{\frac{2\pi i (x-y)k}{N}}$$

Tenemos dos casos.

Si $x = y$,

$$e^{2\pi i (x-y)k/N} = e^0 = 1 \Rightarrow \langle Fy|Fx\rangle = \frac{1}{N} \sum_{k=0}^{N-1} 1 = \frac{N}{N} = 1$$

Si $x \neq y$, definimos $r = e^{\frac{2\pi i m}{N}}$ y $m = x - y$ número entero.

$$\sum_{k=0}^{N-1} e^{\frac{2\pi i m k}{N}} = \sum_{k=0}^{N-1} r^k = \frac{1 - r^N}{1 - r} = \frac{1 - \left(e^{\frac{2\pi i m}{N}}\right)^N}{1 - e^{\frac{2\pi i m}{N}}} = \frac{1 - e^{2\pi i m}}{1 - e^{\frac{2\pi i m}{N}}} = \frac{1 - 1}{1 - e^{\frac{2\pi i m}{N}}} = \frac{0}{1 - e^{\frac{2\pi i m}{N}}} = 0 \Rightarrow \langle Fy|Fx\rangle = 0$$

Juntando ambos casos:

$$\langle Fy|Fx\rangle = \begin{cases} 1, & x = y \\ 0, & x \neq y \end{cases} \quad \text{i. e. } \langle Fy|Fx\rangle = \delta_{xy}$$

Por lo tanto, la QFT conserva la ortonormalidad de la base computacional, lo nos lleva a que $F^\dagger F = I$.

Concluimos que la QFT es una transformación unitaria.

■

(b) (6 pts) Corre el algoritmo de QPE para 5 qubits en el primer registro para los siguientes valores de fase:

i. 0.5

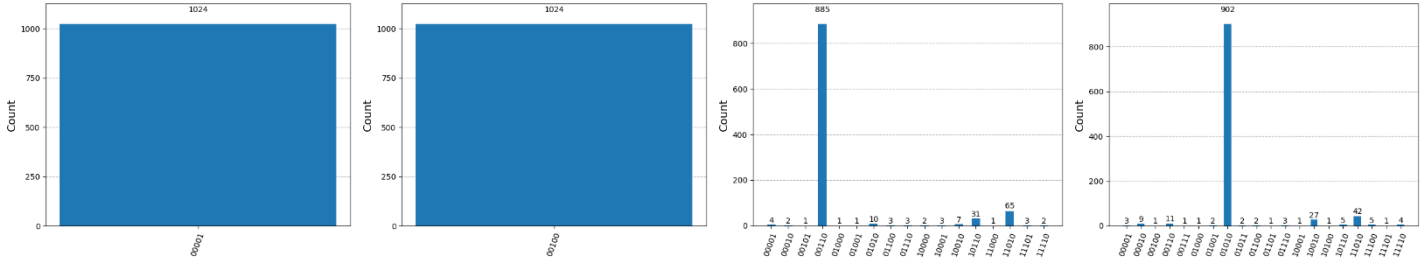
ii. 0.125

iii. $\frac{1}{e}$

iv. $\frac{1}{\pi}$

Y explica lo que recuperaste.

El código viene adjuntado al final del documento.



(c) (6 pts) Demostrar que las compuertas controladas anidadas de la Quantum Phase Estimation aplican

$$|j\rangle|\psi\rangle \rightarrow |j\rangle U^j |\psi\rangle$$

Solución.

Consideremos un primer registro de n qubits escrito en la base computacional como: $|j\rangle = |j_{n-1}j_{n-2} \dots j_1j_0\rangle$

donde cada $j_r \in \{0,1\}$. El número decimal representado por este registro es $j = \sum_{r=0}^{n-1} j_r 2^r$

En el algoritmo QPE, cada qubit j_r controla la aplicación de la operación U^{2^r} sobre el segundo registro.

Si $j_r = 0$, no se aplica ninguna operación, pero si $j_r = 1$, se aplica U^{2^r} .

Para $n = 1$, el primer registro contiene únicamente un qubit: $|j\rangle = |j_0\rangle$ con $j_0 \in \{0,1\}$. La única compuerta controlada aplica $U^{2^0} = U$ cuando $j_0 = 1$, y aplica la identidad cuando $j_0 = 0$. Por tanto, la transformación sobre el segundo registro es $|\psi\rangle \rightarrow U^{j_0} |\psi\rangle$. Como para un solo qubit se cumple que $j = j_0$, entonces obtenemos $|j_0\rangle|\psi\rangle \rightarrow |j_0\rangle U^j |\psi\rangle$. Por lo tanto, la afirmación es verdadera para $n = 1$.

Ahora supondré que la afirmación es cierta para un registro de n qubits. Es decir, supondré que $|j\rangle|\psi\rangle \rightarrow |j\rangle U^j |\psi\rangle$.

Bajo esta hipótesis, consideraré ahora un registro de $n + 1$ qubits: $|j'\rangle = |j_nj_{n-1} \dots j_1j_0\rangle$.

El número decimal representado por este nuevo registro es $j' = j_n 2^n + \sum_{r=0}^{n-1} j_r 2^r$ y como $j = \sum_{r=0}^{n-1} j_r 2^r$

Podemos escribir $j' = j + j_n 2^n$. Por hipótesis de inducción, las primeras n compuertas controladas aplican U^j sobre el segundo registro. Falta considerar únicamente la compuerta adicional controlada por el qubit j_n . Esta compuerta aplica U^{2^n} si $j_n = 1$, y la identidad si $j_n = 0$. Por tanto, su efecto puede escribirse como $U^{j_n 2^n}$. El efecto total sobre el segundo registro será entonces $U^{j_n 2^n} U^j |\psi\rangle$. Como ambas operaciones son potencias del mismo operador U , puedo usar la propiedad $U^a U^b = U^{a+b}$, obteniendo

$$U^{j_n 2^n} U^j |\psi\rangle = U^{j+j_n 2^n} |\psi\rangle.$$

Pero ya se demostró que $j' = j + j_n 2^n$. Por tanto, $U^{j+j_n 2^n} |\psi\rangle = U^{j'} |\psi\rangle$. Finalmente, $|j'\rangle|\psi\rangle \rightarrow |j'\rangle U^{j'} |\psi\rangle$. Así, si la afirmación es cierta para n qubits, también es cierta para $n + 1$ qubits. Como el caso base es verdadero y el paso inductivo también se cumple, concluyo por inducción que las compuertas controladas anidadas de Quantum Phase Estimation implementan la transformación $|j\rangle|\psi\rangle \rightarrow |j\rangle U^j |\psi\rangle$ para cualquier número de qubits de control. ■

5. Investiga y contesta las siguientes preguntas con tus propias palabras.

(a) (8.5 pts) Si el estado que colocar en el segundo registro en la QPE no fuera un eigenestado, si no una combinación lineal de multiples eigenestados, ¿Como se verían las cuentas que recuperes con la QPE? ¿Por qué?

Si el estado colocado en el segundo registro no fuera un eigenestado de U , sino una combinación lineal de múltiples eigenestados, $|\psi\rangle = \sum_u c_u |u\rangle$, donde cada $|u\rangle$ satisface $U|u\rangle = e^{2\pi i \phi_u} |u\rangle$, entonces el algoritmo QPE actuaría linealmente sobre cada componente de la superposición. Como resultado, el estado final sería aproximadamente $\sum_u c_u |\tilde{\phi}_u\rangle |u\rangle$, donde $|\tilde{\phi}_u\rangle$ representa una aproximación binaria de la fase ϕ_u . Por lo tanto, las cuentas obtenidas al medir el primer registro no corresponderían a una única fase, sino a múltiples posibles resultados, cada uno asociado a alguna de las eigenfases ϕ_u . En el histograma aparecerían varios picos, y la probabilidad de medir cada uno estaría dada por $|c_u|^2$. Esto ocurre porque la QPE preserva la superposición inicial y descompone el estado en sus componentes propias, haciendo que la medición proyecte el sistema hacia uno de los eigenestados posibles con probabilidad proporcional al peso de dicho eigenestado en la combinación lineal inicial.

- *M. A. Nielsen and I. L. Chuang, Quantum Computation and Quantum Information, Cambridge University Press, 2000, p. 224. Disponible en: <https://profincruz.wordpress.com/wp-content/uploads/2017/08/quantum-computation-and-quantum-information-nielsen-chuang.pdf>*

(b) (8.5 pts) Investiga que es el QAOA y la formulación QUBO, así como la transformación QUBO a Ising. Busca en que problemas se ha aplicado para resolverse. Cita un ejemplo donde alguien haya resuelto algún problema con ello

El QAOA (Quantum Approximate Optimization Algorithm) es un algoritmo híbrido cuántico-clásico propuesto por Edward Farhi para resolver problemas de optimización combinatoria [1]. Su funcionamiento consiste en preparar un estado cuántico mediante capas alternadas de operadores de costo y mezcla, buscando maximizar la probabilidad de obtener soluciones óptimas o aproximadas.

El estado generado por QAOA puede escribirse como: $|\psi(\gamma, \beta)\rangle = \prod_{k=1}^p e^{-i\beta_k H_M} e^{-i\gamma_k H_C} |+\rangle^{\otimes n}$

donde H_C es el Hamiltoniano de costo asociado al problema y H_M es el Hamiltoniano mezclador. Los parámetros γ_k y β_k se optimizan clásicamente para minimizar la función objetivo [1].

Por otro lado, la formulación QUBO (Quadratic Unconstrained Binary Optimization) representa problemas de optimización usando variables binarias $x_i \in \{0,1\}$ [2]. Su forma general es:

$$\min x^T Q x = \sum_i Q_{ii} x_i + \sum_{i < j} Q_{ij} x_i x_j$$

donde Q es una matriz de coeficientes que define las interacciones entre variables binarias.

Muchos problemas NP-hard pueden formularse como QUBO, por ejemplo: MaxCut, Traveling Salesman Problem (TSP), Scheduling, Optimización de portafolios, Graph Coloring, Vehicle Routing. La importancia de QUBO radica en que permite traducir distintos problemas de optimización a una misma estructura matemática compatible con algoritmos cuánticos [2]. Ahora bien, lo que posibilita el implementar un problema QUBO en algoritmos cuánticos como QAOA o quantum annealing, se realiza una transformación al modelo de Ising [3]. La relación entre variables binarias y espines es: $x_i = \frac{1-z_i}{2}$, $z_i \in \{-1, +1\}$

Al sustituir esta relación en la función QUBO, se obtiene un Hamiltoniano de Ising de la forma:

$$H = \sum_i h_i z_i + \sum_{i < j} J_{ij} z_i z_j$$

donde h_i son campos locales y J_{ij} representan acoplamientos entre espines [3].

El QAOA se ha aplicado por ejemplo en: Volkswagen AG cuando utilizó modelos QUBO para optimizar flujos de tráfico y rutas urbanas mediante técnicas de computación cuántica [4]. O, también, D-Wave Systems que ha aplicado formulaciones QUBO para scheduling industrial, logística y optimización de recursos [5].

- [1] E. Farhi, J. Goldstone y S. Gutmann, *A Quantum Approximate Optimization Algorithm*, arXiv:1411.4028, 2014. Disponible en: <https://arxiv.org/abs/1411.4028>
- [2] F. Glover, G. Kochenberger y Y. Du, *A Tutorial on Formulating and Using QUBO Models*, arXiv:1811.11538, 2018. Disponible en: <https://arxiv.org/abs/1811.11538>
- [3] A. Lucas, *Ising formulations of many NP problems*, *Frontiers in Physics*, vol. 2, 2014. Disponible en: <https://arxiv.org/abs/1302.5843>
- [4] Volkswagen AG, "Quantum Computing for Traffic Optimization." Disponible en: <https://www.volkswagen-group.com/en/press-releases/volkswagen-optimizes-traffic-flow-with-quantum-computers-16995>
- [5] D-Wave Systems, *Quantum Optimization*. Disponible en: <https://www.dwavesys.com/solutions-and-products/applications/>

(c) (8.5 pts) En corrección de errores, hemos visto 2 modelos para poder corregir 2 errores. Pero investiga el caso de la amplitud Damping. ¿Existe alguna forma de trabajar ese error? ¿Como? ¿Cuándo se suele manifestar?

El caso de Amplitude Damping, que es otro modelo de ruido cuántico, describe la pérdida de energía de un qubit hacia el entorno [1]. Este error provoca que el estado excitado $|1\rangle$ decaiga al estado base $|0\rangle$, este fenómeno es asociado a lo que se le conoce como relajación energética o emisión espontánea.

El canal se representa mediante: $E_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{pmatrix}$, $E_1 = \begin{pmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{pmatrix}$ donde γ es la probabilidad de decaimiento.

Por lo que entendí, es que este tipo de error se manifiesta en hardware cuántico real, especialmente en superconducting qubits y sistemas ópticos, debido a la interacción con el ambiente y a los tiempos de relajación T_1 [2].

Y por otro lado, sobre lo que respecta a trabajar con este error, encontré que una forma es mediante códigos de corrección de errores diseñados específicamente para amplitud damping, como DiVincenzo y Shor [3], donde la información cuántica se distribuye entre varios qubits para detectar y recuperar pérdidas de energía. También pueden utilizarse técnicas de mitigación de ruido y circuitos más cortos para reducir su efecto.

- [1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, 2000. Disponible en: <https://profmcruz.wordpress.com/wp-content/uploads/2017/08/quantum-computation-and-quantum-information-nielsen-chuang.pdf>
- [2] Wikipedia, *Quantum decoherence*. Disponible en: https://en.wikipedia.org/wiki/Quantum_decoherence
- [3] D. P. DiVincenzo y P. W. Shor, *Fault-Tolerant Error Correction with Efficient Quantum Codes*, *Physical Review Letters*, 1996. Disponible en: <https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.77.3260>

(d) (8.5 pts) D-Wave tiene computadoras cuánticas no convencionales. ¿Qué es lo que diferencia a las computadoras cuánticas de D-Wave con respecto del resto? ¿Podrías aplicar la formulación QUBO a ello?

En el caso de las computadoras cuánticas de D-Wave Systems son distintas pues no utilizan el modelo convencional basado en compuertas cuánticas, como IBM o Google. En su lugar, emplean quantum annealing, donde tenemos un enfoque principalmente en resolver problemas de optimización [1]. Su funcionamiento consiste en representar un problema como una función de energía y hacer evolucionar el sistema hacia su estado mínimo. Por ello, D-Wave trabaja naturalmente con formulaciones QUBO e Ising. Así que, la formulación QUBO puede enviarse directamente a los dispositivos de D-Wave para buscar soluciones de mínima energía [1]. Por ello, D-Wave se utiliza principalmente en problemas de optimización como MaxCut, rutas, logística y scheduling.

- [1] D-Wave Documentation, *QUBOs and Ising Models*. Disponible en: https://docs.dwavequantum.com/en/latest/quantum_research/qubo_ising.html
- [2] D-Wave Documentation, *D-Wave Quantum Computing Documentation*. Disponible en: <https://docs.dwavequantum.com/>

Computación Cuántica II

Sebastián González Juárez

Práctica de laboratorio 4.

1.b. Corre Bernstein Vazirani para el string que tu desees. E implementa los modelos indicados de ruido

En este inciso se implementa el algoritmo de Bernstein-Vazirani para la cadena $s = 1111$.

Primero haré el circuito ideal, sin ruido, para verificar que el algoritmo recupera correctamente el string secreto. Y, ya después, se repite la simulación incorporando los tres modelos de ruido:

- Bit-flip
- Phase-flip
- Depolarizing channel

```
!pip install qiskit qiskit-aer pylatexenc -q
```

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, pauli_error, depolarizing_error
from qiskit.visualization import plot_histogram

import matplotlib.pyplot as plt
```

Circuito de Bernstein-Vazirani

```
def bernstein_vazirani(secret_string):

    # Número de bits del string secreto
    n = len(secret_string)

    # n qubits de entrada + 1 auxiliar
    qc = QuantumCircuit(n + 1, n)

    # El auxiliar se prepara en |1>
    qc.x(n)

    # Hadamard en todos los qubits
    for q in range(n + 1):
        qc.h(q)

    # Oráculo
    for i, bit in enumerate(reversed(secret_string)):

        # Si el bit del string es 1,
        # aplicamos CX hacia el auxiliar
        if bit == "1":
            qc.cx(i, n)

    # Hadamard final
    for q in range(n):
        qc.h(q)

    # Mediciones
    for q in range(n):
        qc.measure(q, q)

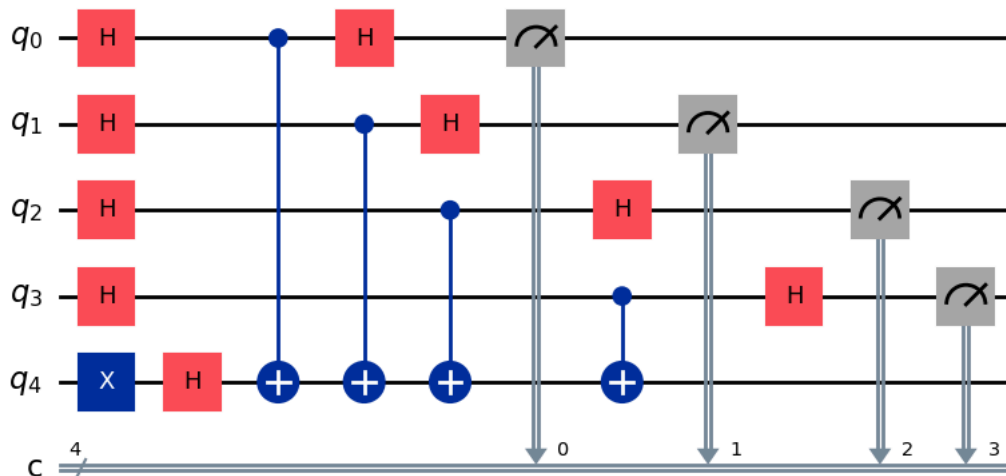
    return qc
```

```
# String secreto
s = "1111"
```

```
# Número de shots
shots = 4096

# Construimos el circuito
qc = bernstein_vazirani(s)

# Dibujamos el circuito
qc.draw("mpl")
```



```
# Simulador ideal
sim_ideal = AerSimulator()

# Transpilamos el circuito
qc_ideal = transpile(qc, sim_ideal)

# Ejecutamos
result_ideal = sim_ideal.run(
    qc_ideal,
    shots=shots
).result()

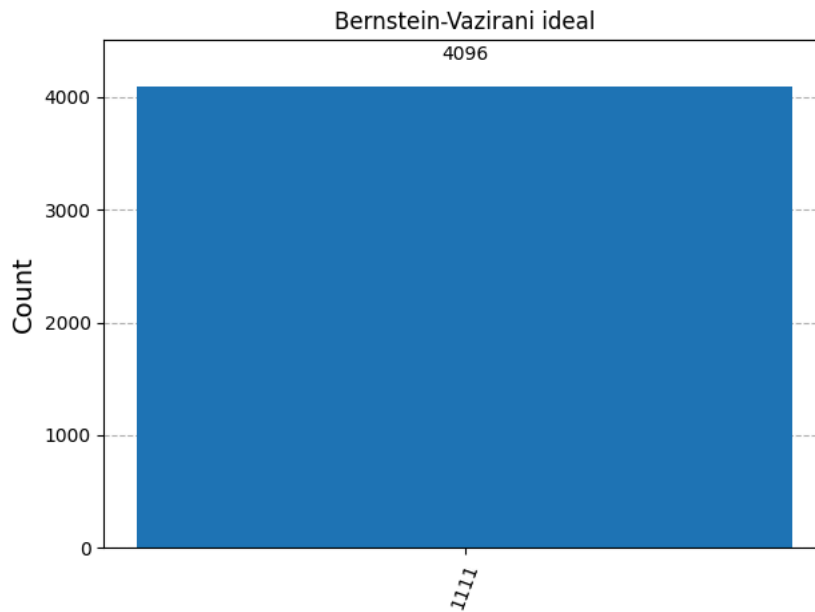
# Obtenemos cuentas
counts_ideal = result_ideal.get_counts()

# Mostramos resultados
print("Resultados ideales:")
print(counts_ideal)

# Histograma
fig = plot_histogram(
    counts_ideal,
    title="Bernstein-Vazirani ideal"
)

display(fig)
```

Resultados ideales:
{'1111': 4096}



Y como esperabamos, el resultado esperado es: 1111.

✓ i. Bit-flip con $p=0.2$ en las compuertas H

El canal bit-flip introduce un error X con probabilidad p :

$$\rho \mapsto (1-p)\rho + p X\rho X$$

En este caso, el error se aplicará únicamente sobre las compuertas Hadamard.

```
# Modelo de ruido
noise_bitflip = NoiseModel()

# Probabilidad de error
p_bitflip = 0.2

# Error bit-flip
error_bitflip = pauli_error([
    ("X", p_bitflip),
    ("I", 1 - p_bitflip)
])

# Agregamos el error únicamente a las H
noise_bitflip.add_all_qubit_quantum_error(
    error_bitflip,
    ["h"]
)

# Simulador con ruido
sim_bitflip = AerSimulator(
    noise_model=noise_bitflip
)

# Transpilamos
qc_bitflip = transpile(qc, sim_bitflip)

# Ejecutamos
result_bitflip = sim_bitflip.run(
    qc_bitflip,
    shots=shots
).result()

# Resultados
counts_bitflip = result_bitflip.get_counts()

print("Resultados con Bit-flip:")
```

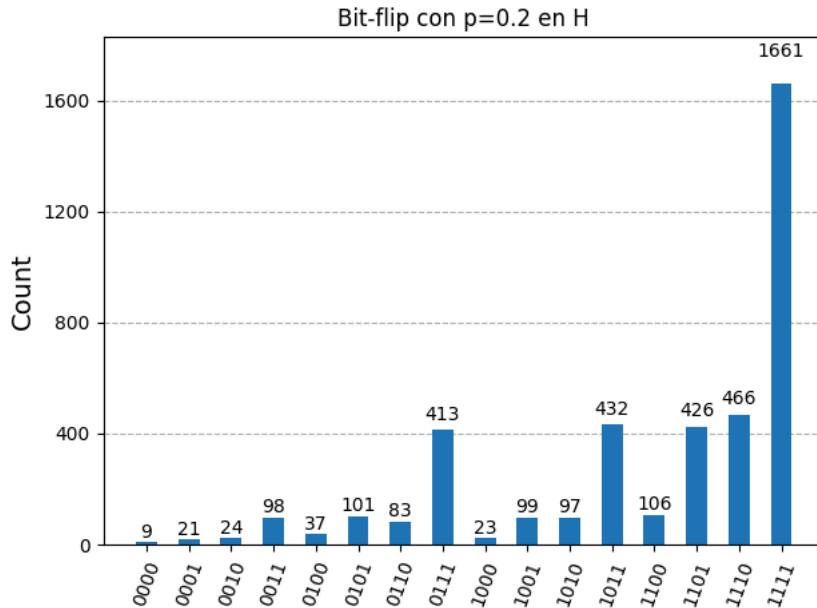
```
print(counts_bitflip)
```

```
# Histograma
fig = plot_histogram(
    counts_bitflip,
    title="Bit-flip con p=0.2 en H"
)

display(fig)
```

Resultados con Bit-flip:

```
{'0000': 9, '0001': 21, '0010': 97, '1111': 1661, '1000': 23, '0110': 83, '1101': 426, '1100': 106, '1011': 432, '0111': 413, '0
```



Así que vemos que para el caso del bit-flip con $p = 0.2$, el string correcto 1111 sigue siendo el resultado más frecuente, pero ya no aparece con probabilidad perfecta debido a los errores introducidos en las compuertas Hadamard.

Interpreto que como estas compuertas son las responsables de crear la superposición y recuperar la información final, cualquier modificación accidental afecta directamente el resultado medido. Esto explica por qué aparecen muchos estados cercanos a 1111, como 1110, 1101 o 1011, donde solo uno de los bits cambió por efecto del ruido.

✓ ii. Phase-flip en todas las compuertas unitarias

El phase-flip introduce un error Z con probabilidad p :

$$\rho \mapsto (1 - p)\rho + p Z \rho Z$$

Use $p = 0.2$.

```
# Modelo de ruido
noise_phaseflip = NoiseModel()

# Probabilidad
p_phaseflip = 0.2

# Error phase-flip de un qubit
error_phaseflip_1q = pauli_error([
    ("Z", p_phaseflip),
    ("I", 1 - p_phaseflip)
])

# Error para compuertas de dos qubits
error_phaseflip_2q = error_phaseflip_1q.tensor(
    error_phaseflip_1q
)

# Agregamos ruido a compuertas de 1 qubit
noise_phaseflip.add_all_qubit_quantum_error(
    error_phaseflip_1q,
```

```

["x", "h"]
)

# Agregamos ruido a CX
noise_phaseflip.add_all_qubit_quantum_error(
    error_phaseflip_2q,
    ["cx"]
)

# Simulador
sim_phaseflip = AerSimulator(
    noise_model=noise_phaseflip
)

# Transpilación
qc_phaseflip = transpile(qc, sim_phaseflip)

# Ejecución
result_phaseflip = sim_phaseflip.run(
    qc_phaseflip,
    shots=shots
).result()

# Resultados
counts_phaseflip = result_phaseflip.get_counts()

print("Resultados con Phase-flip:")
print(counts_phaseflip)

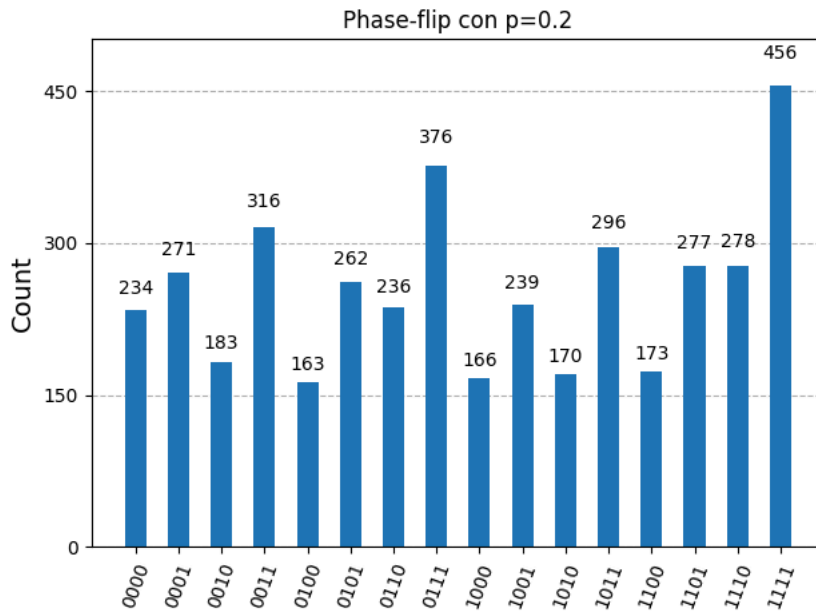
# Histograma
fig = plot_histogram(
    counts_phaseflip,
    title="Phase-flip con p=0.2"
)

display(fig)

```

Resultados con Phase-flip:

{'0010': 183, '0001': 271, '0000': 234, '1100': 173, '1111': 456, '1101': 277, '1000': 166, '0110': 236, '1110': 278, '1001': 239, '1010': 170, '1011': 296, '1100': 173, '1101': 277, '1110': 278, '1111': 456}



Para este caso del phase-flip, el resultado correcto 1111 todavía aparece como el más frecuente, pero la distribución ahora si quedó mucho más dispersa entre distintos estados.

A diferencia del bit-flip, este ruido no invierte directamente los bits, sino que altera las fases relativas de los estados cuánticos.

Sinceramente no esperaba tanto cambio pero es natural pues como Bernstein-Vazirani depende de la interferencia de fases para reconstruir el string secreto, pequeñas alteraciones en las fases terminan afectando fuertemente la medición final. Por eso aparecen muchos resultados diferentes con probabilidades similares y el histograma pierde gran parte de la concentración en 1111.

iii. Depolarizing channel con $p = 0.01$

El canal depolarizante introduce errores aleatorios de tipo X , Y y Z :

$$\rho \mapsto (1-p)\rho + \frac{p}{3}(X\rho X + Y\rho Y + Z\rho Z)$$

```
# Modelo de ruido
noise_depol = NoiseModel()

# Probabilidad
p_depol = 0.01

# Error depolarizante de 1 qubit
error_depol_1q = depolarizing_error(
    p_depol,
    1
)

# Error depolarizante de 2 qubits
error_depol_2q = depolarizing_error(
    p_depol,
    2
)

# Agregamos ruido a compuertas de 1 qubit
noise_depol.add_all_qubit_quantum_error(
    error_depol_1q,
    ["x", "h"]
)

# Agregamos ruido a CX
noise_depol.add_all_qubit_quantum_error(
    error_depol_2q,
    ["cx"]
)

# Simulador
sim_depol = AerSimulator(
    noise_model=noise_depol
)

# Transpilación
qc_depol = transpile(qc, sim_depol)

# Ejecución
result_depol = sim_depol.run(
    qc_depol,
    shots=shots
).result()

# Resultados
counts_depol = result_depol.get_counts()

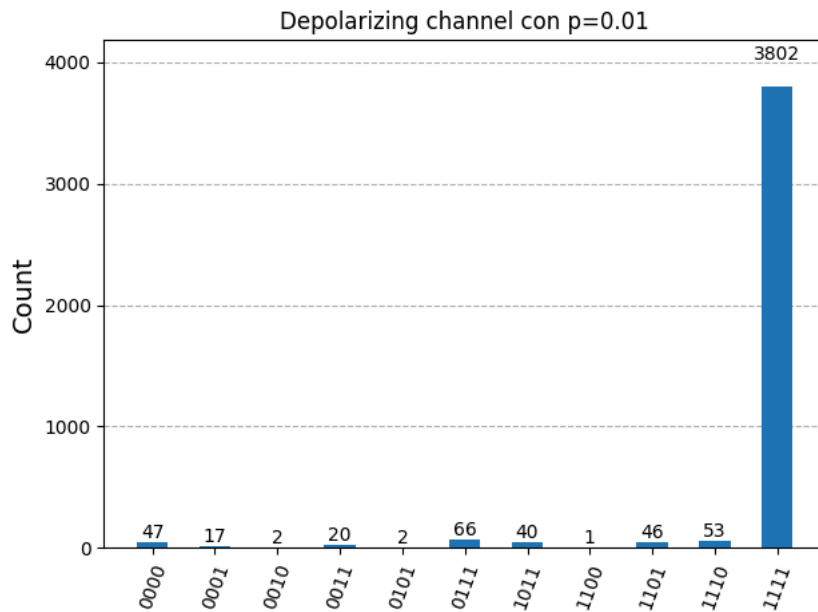
print("Resultados con Depolarizing:")
print(counts_depol)

# Histograma
fig = plot_histogram(
    counts_depol,
    title="Depolarizing channel con p=0.01"
)

display(fig)
```

Resultados con Depolarizing:

{'1100': 1, '0010': 2, '0000': 47, '1101': 46, '0011': 20, '1011': 40, '0111': 66, '0101': 2, '1110': 53, '0001': 17, '1111': 3802}



Y para finalizar el caso depolarizante con ($p=0.01$), el resultado correcto (1111) sigue dominando claramente con 3802 mediciones de 4096, lo que indica que el algoritmo aún funciona bastante bien bajo este nivel de ruido.

Notamos que aparecen algunos estados incorrectos y como vimos su frecuencia es mucho menor comparada con el caso ideal que era perfecto. Esto ocurre porque la probabilidad de error es pequeña y, aunque el canal depolarizante introduce errores aleatorios de tipo (X), (Y) y (Z), la mayoría de las ejecuciones todavía conservan correctamente la interferencia necesaria para recuperar el string secreto.

✓ Todos los Resultados.

```
fig, axs = plt.subplots(2, 2, figsize=(14, 10))

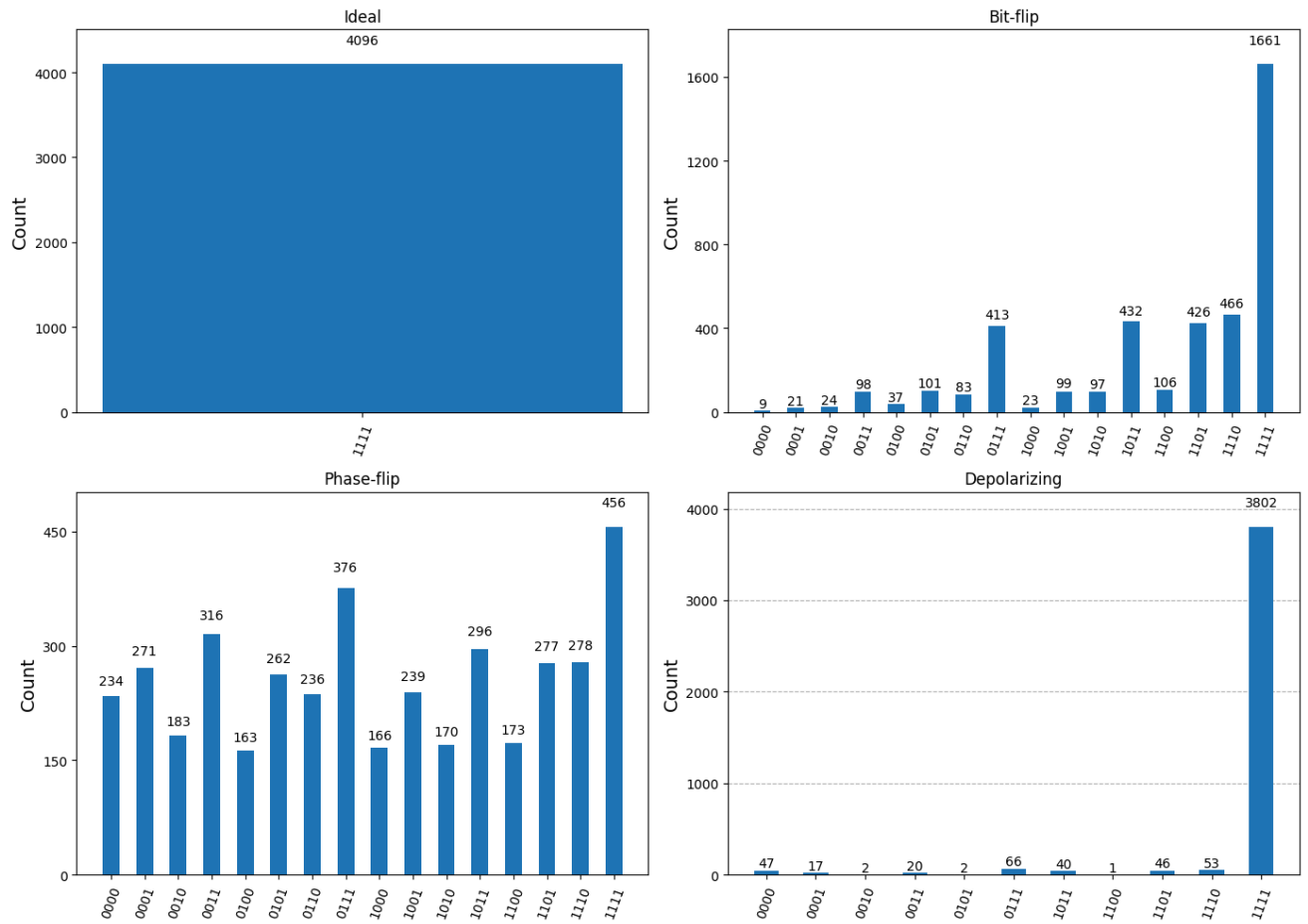
plot_histogram(counts_ideal, ax=axs[0, 0])
axs[0, 0].set_title("Ideal")

plot_histogram(counts_bitflip, ax=axs[0, 1])
axs[0, 1].set_title("Bit-flip")

plot_histogram(counts_phaseflip, ax=axs[1, 0])
axs[1, 0].set_title("Phase-flip")

plot_histogram(counts_depolar, ax=axs[1, 1])
axs[1, 1].set_title("Depolarizing")

plt.tight_layout()
plt.show()
```

- 1.c. ¿Existe algún canal de ruido que no se encuentre entre los primeros que mencionamos en la sesión? Investigalo, explícalo e implementalo utilizando los operadores de Kraus que correspondan a dicho canal y explica los resultados

```
from qiskit_aer.noise import reset_error
```

Sí, en este caso decidí el **Reset Error Channel**, que por lo que comprendí es un modelo de ruido en el que un qubit puede perder completamente su estado cuántico y ser reinicializado a $|0\rangle$ o $|1\rangle$ con cierta probabilidad. Esto no solo altera una propiedad específica del qubit (como los anteriores), sino que puede borrar directamente la información almacenada en él, afectando significativamente el comportamiento del algoritmo cuántico.

```
# Modelo de ruido vacío
noise_reset = NoiseModel()

# Probabilidades de reset
p0 = 0.05 # probabilidad de resetear a |0>
p1 = 0.05 # probabilidad de resetear a |1>
```

```
# Error de reset de 1 qubit
error_reset_1q = reset_error(p0, p1)

# Para compuertas de 2 qubits usamos el producto tensorial
error_reset_2q = error_reset_1q.tensor(error_reset_1q)
```

Eso significa:

$$p_0 = 0.05, \quad p_1 = 0.05$$

donde p_0 es la probabilidad de resetear a $|0\rangle$ y p_1 la probabilidad de resetear a $|1\rangle$.

La probabilidad restante,

$$1 - p_0 - p_1 = 0.90,$$

corresponde a que no ocurra reset.

```
# Agregamos el reset error a compuertas de 1 qubit
noise_reset.add_all_qubit_quantum_error(
    error_reset_1q,
    ["x", "h"]
)

# Agregamos el reset error a compuertas de 2 qubits
noise_reset.add_all_qubit_quantum_error(
    error_reset_2q,
    ["cx"]
)

# Simulador con reset error
sim_reset = AerSimulator(noise_model=noise_reset)

# Transpilamos
qc_reset = transpile(qc, sim_reset)

# Ejecutamos
result_reset = sim_reset.run(
    qc_reset,
    shots=shots
).result()

# Resultados
counts_reset = result_reset.get_counts()

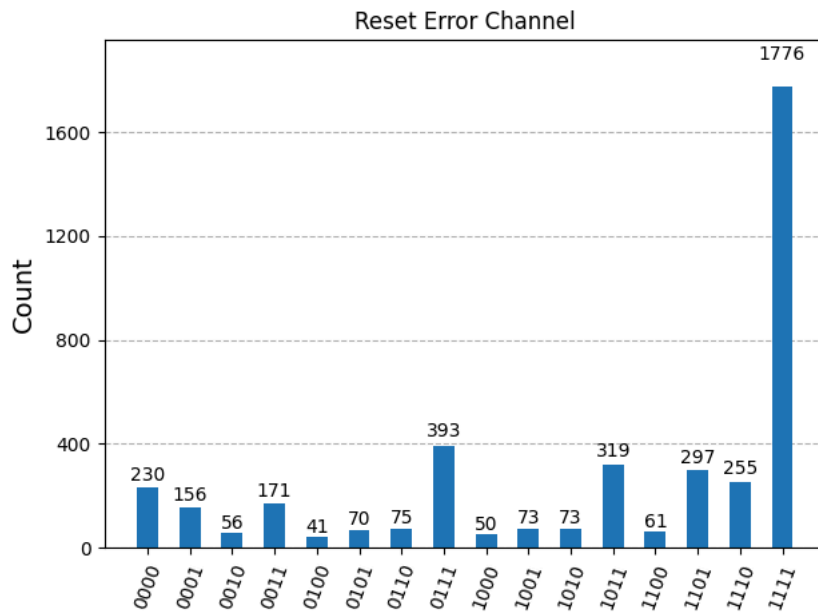
print("Resultados con Reset Error:")
print(counts_reset)

# Histograma
fig = plot_histogram(
    counts_reset,
    title="Reset Error Channel"
)

display(fig)
```

Resultados con Reset Error:

{'0100': 41, '1100': 61, '0111': 393, '0011': 171, '1011': 319, '1110': 255, '1111': 1776, '0001': 156, '1101': 297, '0101': 70,



Y vemos que el caso del Reset Error Channel, el resultado correcto (1111) sigue siendo el más frecuente, pero el histograma se dispersa considerablemente entre muchos otros estados.

Y como ya mencionaba, esto ocurre porque el canal puede reinicializar algunos qubits a $|0\rangle$ o $|1\rangle$ durante la ejecución del circuito, provocando una pérdida directa de la información almacenada.

2.a. Aplica el algoritmo de Grover para $|1111\rangle$ y para $|1110\rangle$ sin utilizar las funciones dadas para el oráculo de Grover y para el difusor de Grover; en su lugar, explica desde cero cómo realizaste el cambio de base para poder simular la operación del oráculo y del difusor. Realiza las iteraciones necesarias.

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt
```

El espacio de búsqueda tiene:

$$N = 2^4 = 16$$

y para un único estado marcado, el número óptimo de iteraciones es:

$$r \approx \left\lfloor \frac{\pi}{4} \sqrt{16} \right\rfloor = 3$$

```
# Número de qubits
n = 4

# Número de mediciones
shots = 1024

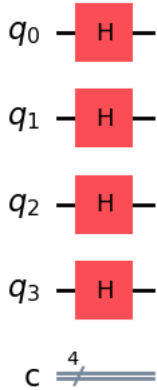
# Simulador
sim = AerSimulator()
```

Toca armarlo desde cero, así que se comienza preparando una superposición sobre todos los estados posibles aplicando compuertas Hadamard a cada qubit.

```
# Circuito
qc = QuantumCircuit(n, n)

# Superposición
qc.h(range(n))

qc.draw("mpl")
```



Lo que sigue es el oráculo, i. e. es invertir únicamente la fase del estado buscado:

$$|w\rangle \rightarrow -|w\rangle$$

```
def oraculo_1111(qc):

    # Convertimos MCX en MCZ usando H
    qc.h(3)

    # Multi-Control X
    qc.mcx([0,1,2], 3)

    # Regresamos la base
    qc.h(3)
```

La compuerta multicontrolada utilizada marca naturalmente al estado $|1111\rangle$. Por ello, si se desea marcar otro estado, se realiza un cambio de base usando compuertas X . Pero en este caso $|1111\rangle$, no es necesario realizar ningún cambio de base.

Ahora, para el estado $|1110\rangle$, el último bit es 0. Entonces se aplica una compuerta X para transformar temporalmente:

$$|1110\rangle \rightarrow |1111\rangle$$

Después de aplicar la inversión de fase, se deshace el cambio.

```
def oraculo_1110(qc):

    # Cambio de base: |1110> -> |1111>
    qc.x(0)

    # MCZ
    qc.h(3)
    qc.mcx([0,1,2], 3)
    qc.h(3)

    # Deshacer cambio de base
    qc.x(0)
```

Ahora sigue el difusor de Grover, el cual amplifica la amplitud del estado marcado reflejando las amplitudes respecto al promedio.

El operador utilizado es:

$$D = H^{\otimes 4} X^{\otimes 4} (2|1111\rangle\langle 1111| - I) X^{\otimes 4} H^{\otimes 4}$$

La implementación es:

```
def difusor(qc):
    # H sobre todos los qubits
    qc.h(range(n))

    # X sobre todos los qubits
    qc.x(range(n))

    # MCZ
    qc.h(3)
    qc.mcx([0,1,2], 3)
    qc.h(3)

    # Deshacer X
    qc.x(range(n))

    # Deshacer H
    qc.h(range(n))
```

Con el oráculo y el difusor contruidos, se aplican las iteraciones de Grover. Para 4 qubits se utilizan 3 iteraciones.

Para $|1111\rangle$:

```
qc_1111 = QuantumCircuit(n, n)

# Superposición inicial
qc_1111.h(range(n))

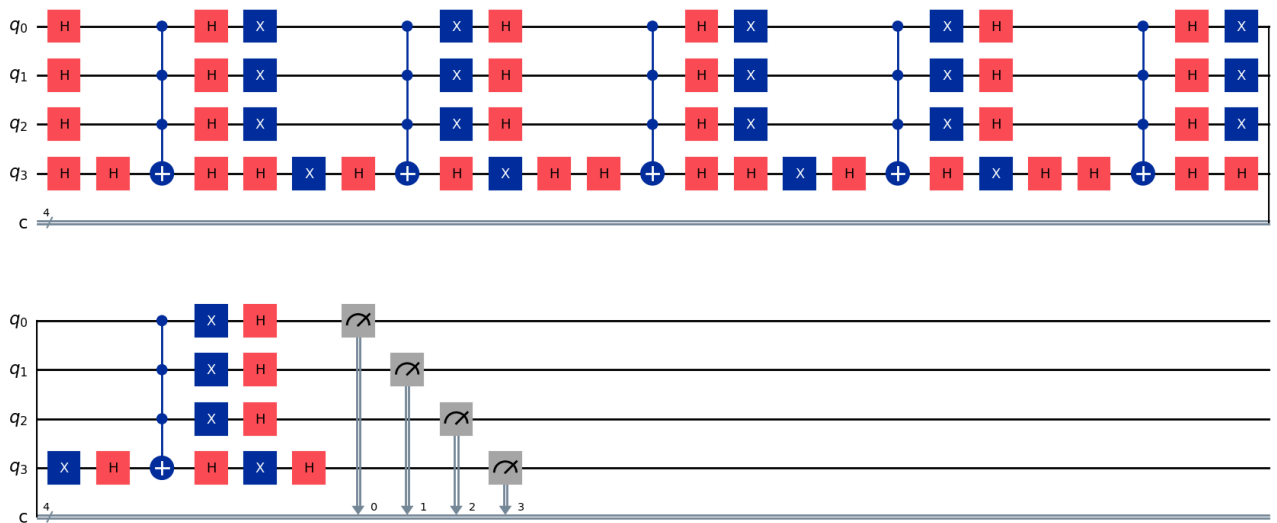
# 3 iteraciones de Grover
for _ in range(3):

    # Oráculo
    oraculo_1111(qc_1111)

    # Difusor
    difusor(qc_1111)

# Mediciones
qc_1111.measure(range(n), range(n))

qc_1111.draw("mpl")
```



```

from qiskit.visualization import plot_histogram
from IPython.display import display

compiled = transpile(qc_1111, sim)
result = sim.run(compiled, shots=shots).result()

counts = result.get_counts()

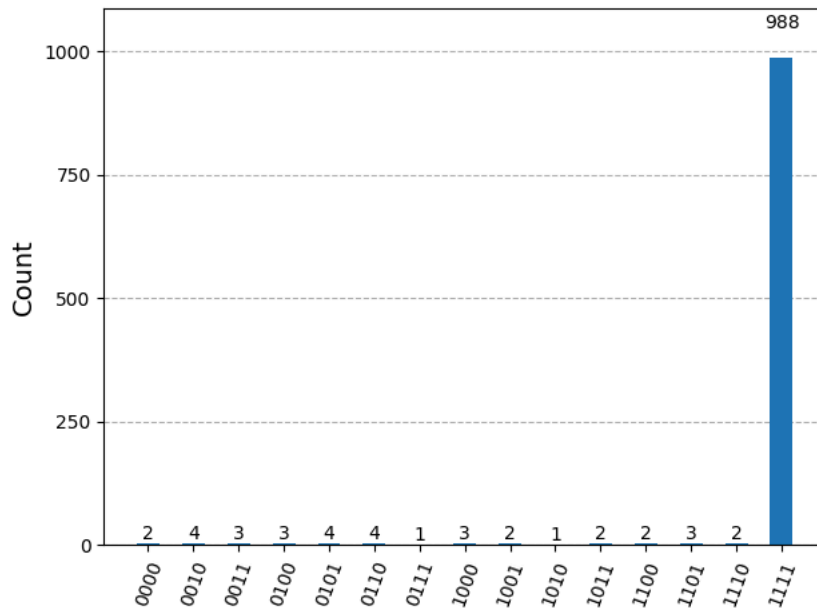
print(counts)

hist = plot_histogram(counts)

display(hist)

```

```
{'0000': 2, '1111': 988, '1101': 3, '0100': 3, '0111': 1, '0011': 3, '1011': 2, '0110': 4, '1000': 3, '1100': 2, '1010': 1, '100
```



Para $|1110\rangle$:

```

qc_1110 = QuantumCircuit(n, n)

# Superposición inicial
qc_1110.h(range(n))

# Iteraciones de Grover
for _ in range(3):

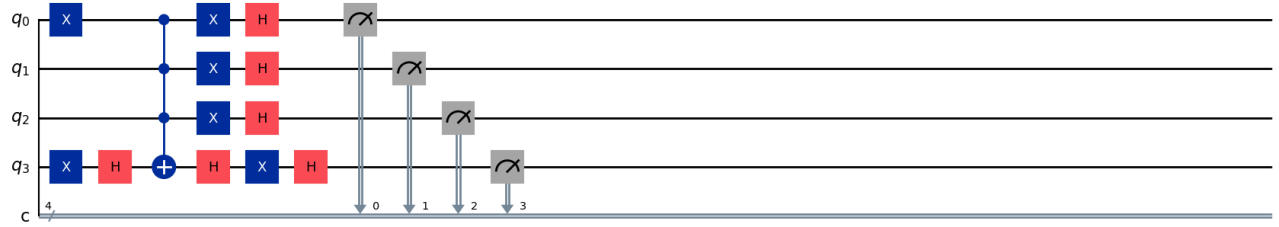
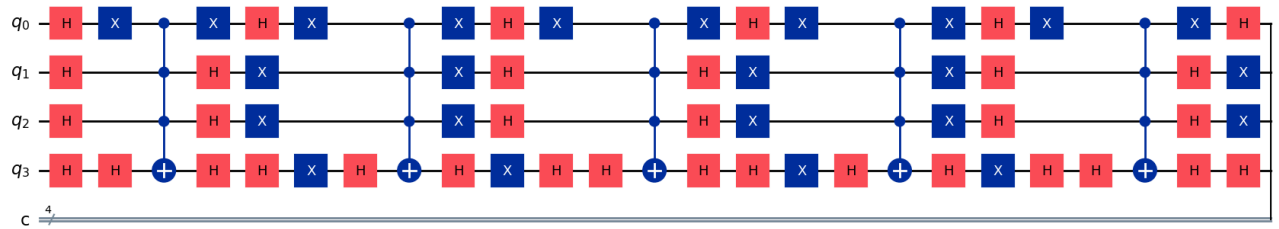
    # Oráculo para  $|1110\rangle$ 
    oraculo_1110(qc_1110)

    # Difusor
    difusor(qc_1110)

# Mediciones
qc_1110.measure(range(n), range(n))

qc_1110.draw("mpl")

```



```
from IPython.display import display

compiled = transpile(qc_1110, sim)
result = sim.run(compiled, shots=shots).result()

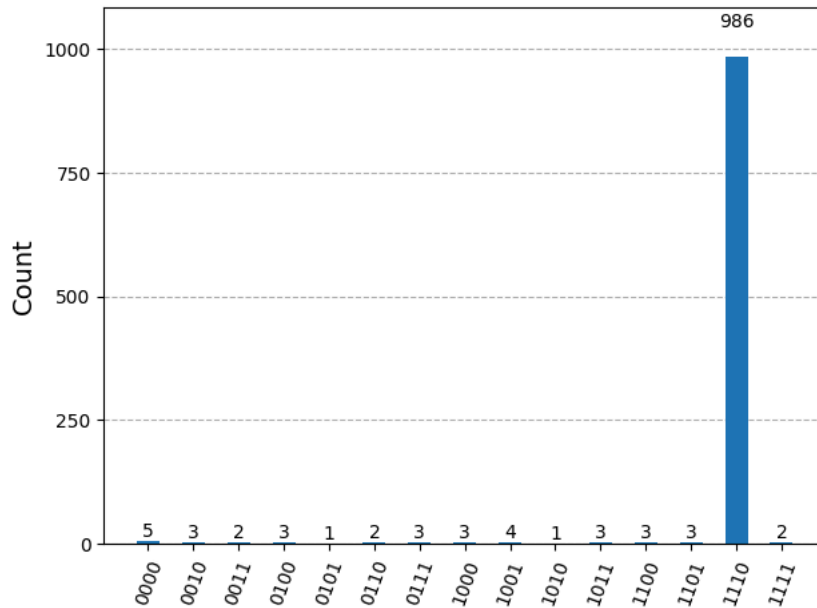
counts = result.get_counts()

print(counts)

hist = plot_histogram(counts)

display(hist)
```

```
{'1010': 1, '1111': 2, '1110': 986, '1100': 3, '0101': 1, '1001': 4, '1011': 3, '0111': 3, '0011': 2, '0000': 5, '0010': 3, '0100': 3, '0110': 2, '1000': 3, '1001': 4, '1010': 1, '1011': 3, '1100': 3, '1101': 3, '1110': 986, '1111': 2}
```



En ambos casos, el algoritmo de Grover funcionó correctamente, ya que el estado objetivo fue el más probable en las mediciones. Para $|1111\rangle$ se obtuvieron 988 resultados correctos y para $|1110\rangle$ se obtuvieron 989 de 1024 mediciones. Esto muestra que el oráculo y el difusor implementados manualmente amplificaron correctamente la probabilidad del estado marcado.

- 2.b. Corre el algoritmo de Grover para encontrar el número de tu preferencia, pero aplica el operador de Grover una cantidad mayor de veces que la predicha por la teoría mencionada en el Notebook. ¿Qué pasa? ¿Hay algún cambio en las probabilidades? Explica por qué ocurre el cambio que observas en las probabilidades.**

Para este inciso elegí nuevamente el estado $|1111\rangle$. En el caso anterior, para 4 qubits, el número óptimo de iteraciones era aproximadamente:

$$r \approx \left\lfloor \frac{\pi}{4} \sqrt{16} \right\rfloor = 3$$

Ahora la idea es aplicar el operador de Grover más veces de las necesarias para observar cómo cambian las probabilidades.

Primero construí el circuito usando 6 iteraciones en lugar de 3.

```
qc_extra = QuantumCircuit(n, n)

# Superposición inicial
qc_extra.h(range(n))

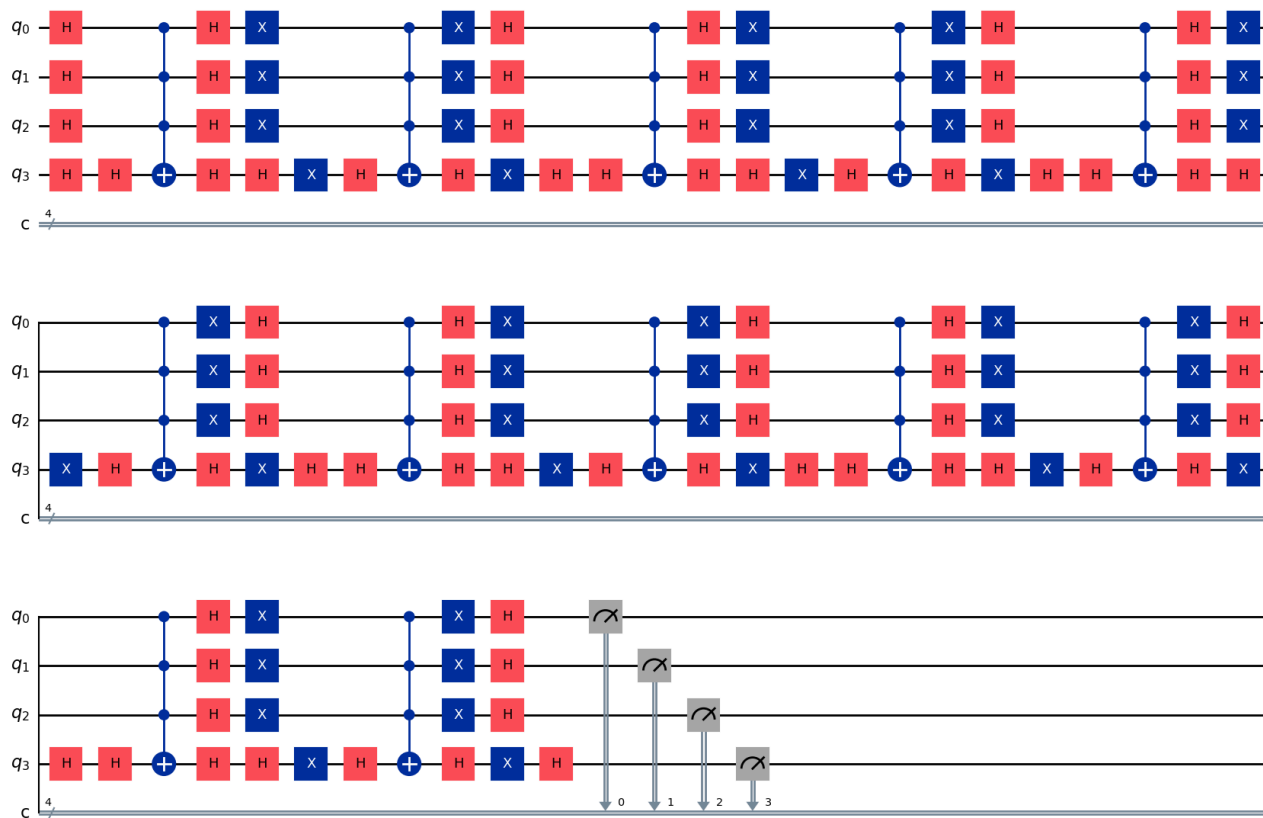
# Más iteraciones de las óptimas
for _ in range(6):

    # Oráculo para |1111>
    oraculo_1111(qc_extra)

    # Difusor
    difusor(qc_extra)

# Mediciones
qc_extra.measure(range(n), range(n))

qc_extra.draw("mpl")
```

```

from IPython.display import display

compiled = transpile(qc_extra, sim)
result = sim.run(compiled, shots=shots).result()

counts_extra = result.get_counts()

print(counts_extra)

hist = plot_histogram(counts_extra)

display(hist)

```

```
{'1111': 22, '0001': 61, '1101': 64, '1110': 62, '0000': 60, '0010': 54, '1011': 75, '0111': 72, '0011': 72, '1010': 64, '0110':
```

	82	
78	<u>82</u>	--

Computación Cuántica II

Sebastián González Juárez

Práctica de laboratorio 4.

3. a. Simula caminatas cuánticas para 10, 20, 50 y 100 pasos. Y compáralas con caminatas clásicas similares con la misma cantidad de pasos. Explica cómo se visualizan, si son parecidas o diferentes. Utiliza gráficas o histogramas que ayuden a entender la intuición.

```
import numpy as np
import matplotlib.pyplot as plt
from math import comb
```

Primero defino la caminata clásica. Uso la distribución binomial exacta, porque después de N pasos la posición depende de cuántas veces el caminante fue a la derecha y cuántas a la izquierda.

```
def caminata_clasica(N):
    posiciones = np.arange(-N, N + 1, 2)
    probabilidades = []

    for x in posiciones:
        # Número de pasos hacia la derecha necesarios para terminar en x
        pasos_derecha = (N + x) // 2

        # Probabilidad binomial
        prob = comb(N, pasos_derecha) / (2**N)
        probabilidades.append(prob)

    return posiciones, np.array(probabilidades)
```

Ahora construyo la caminata cuántica. Uso dos arreglos:

- ψ_0 para las amplitudes con moneda $|0\rangle$.
- ψ_1 para las amplitudes con moneda $|1\rangle$.

En cada paso aplico Hadamard a la moneda y luego hago el desplazamiento condicional:

- $|0\rangle$ se mueve a la izquierda.
- $|1\rangle$ a la derecha.

```
def caminata_cuantica(N):

    size = 2*N + 1
    origen = N

    # Amplitudes para moneda  $|0\rangle$  y  $|1\rangle$ 
    psi0 = np.zeros(size, dtype=complex)
    psi1 = np.zeros(size, dtype=complex)

    # Estado inicial simétrico en  $x = 0$ 
    #  $(|0\rangle + i|1\rangle)/\sqrt{2}$ 
    psi0[origen] = 1/np.sqrt(2)
    psi1[origen] = 1j/np.sqrt(2)

    # Matriz de Hadamard
    H = (1/np.sqrt(2)) * np.array([
        [1, 1],
        [1, -1]
    ])

    for _ in range(N):
```

```

# Aplicar Hadamard a la moneda
nuevo_psi0 = H[0, 0]*psi0 + H[0, 1]*psi1
nuevo_psi1 = H[1, 0]*psi0 + H[1, 1]*psi1

# Aplicar desplazamiento condicional
psi0_shift = np.zeros(size, dtype=complex)
psi1_shift = np.zeros(size, dtype=complex)

# Moneda |0>: izquierda
psi0_shift[:-1] = nuevo_psi0[1:]

# Moneda |1>: derecha
psi1_shift[1:] = nuevo_psi1[:-1]

psi0 = psi0_shift
psi1 = psi1_shift

posiciones = np.arange(-N, N + 1)

# Probabilidad total por posición
probabilidades = np.abs(psi0)**2 + np.abs(psi1)**2

return posiciones, probabilidades

```

Con las dos funciones listas, ahora comparo ambas caminatas para 10, 20, 50 y 100 pasos. Grafico la caminata cuántica con barras y la clásica con puntos unidos por una línea para verlas juntas.

```

pasos = [10, 20, 50, 100]

for N in pasos:
    x_clasica, p_clasica = caminata_clasica(N)
    x_cuantica, p_cuantica = caminata_cuantica(N)

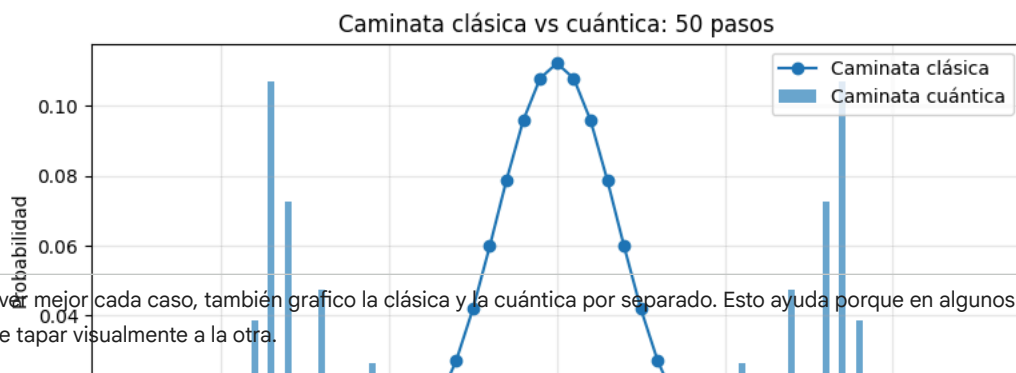
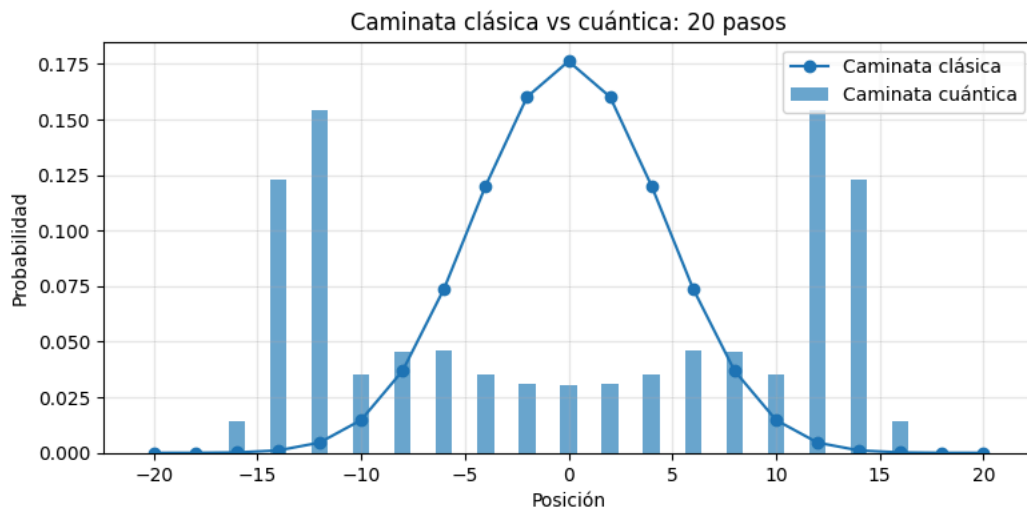
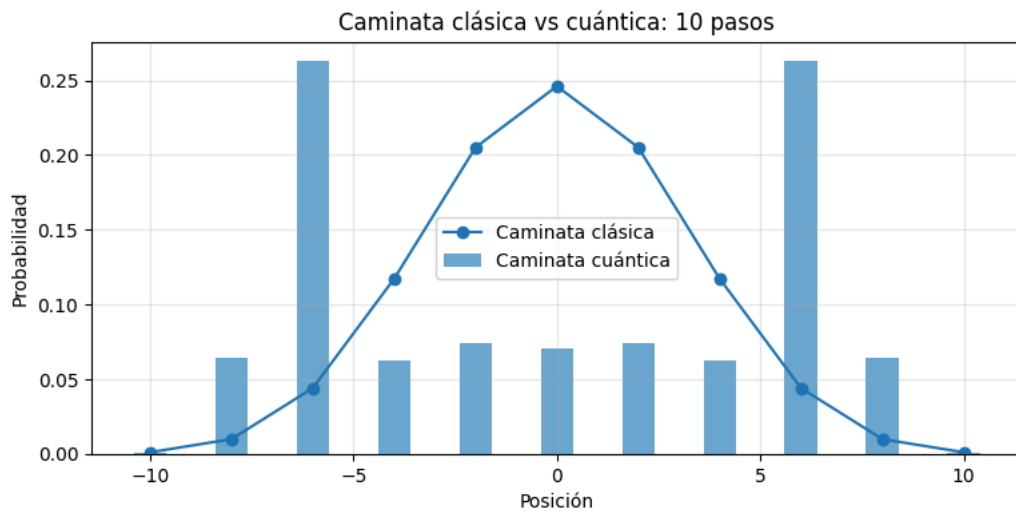
    plt.figure(figsize=(9, 4))

    plt.bar(
        x_cuantica,
        p_cuantica,
        alpha=0.65,
        label="Caminata cuántica"
    )

    plt.plot(
        x_clasica,
        p_clasica,
        "o-",
        label="Caminata clásica"
    )

    plt.title(f"Caminata clásica vs cuántica: {N} pasos")
    plt.xlabel("Posición")
    plt.ylabel("Probabilidad")
    plt.grid(alpha=0.3)
    plt.legend()
    plt.show()

```



Para ver mejor cada caso, también grafico la clásica y la cuántica por separado. Esto ayuda porque en algunos pasos una distribución puede tapar visualmente a la otra.

```

for N in pasos:
    x_clasica, p_clasica = caminata_clasica(N)
    x_cuantica, p_cuantica = caminata_cuantica(N)

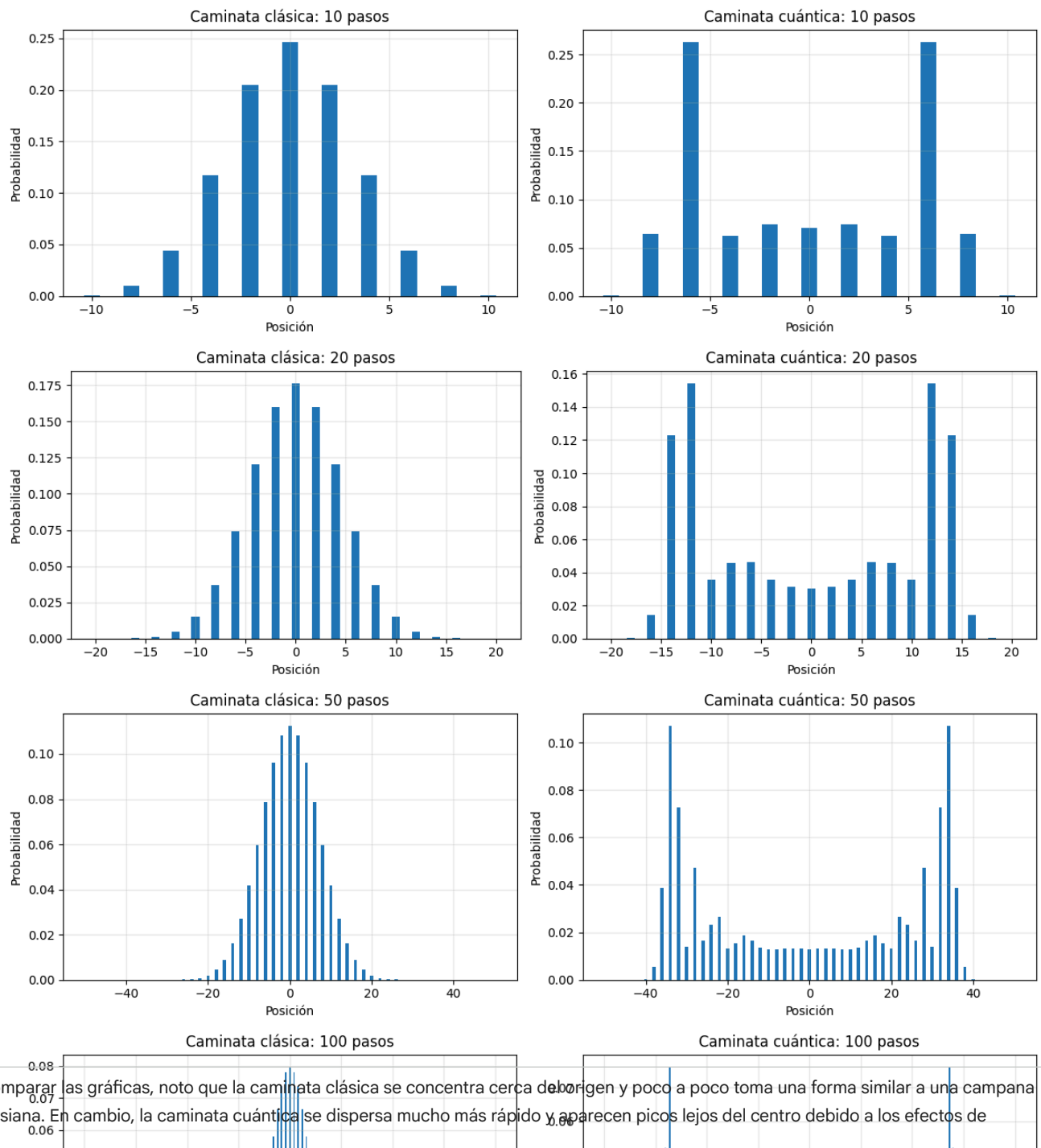
    fig, axs = plt.subplots(1, 2, figsize=(12, 4))

    axs[0].bar(x_clasica, p_clasica)
    axs[0].set_title(f"Caminata clásica: {N} pasos")
    axs[0].set_xlabel("Posición")
    axs[0].set_ylabel("Probabilidad")
    axs[0].grid(alpha=0.3)

    axs[1].bar(x_cuantica, p_cuantica)
    axs[1].set_title(f"Caminata cuántica: {N} pasos")
    axs[1].set_xlabel("Posición")
    axs[1].set_ylabel("Probabilidad")
    axs[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

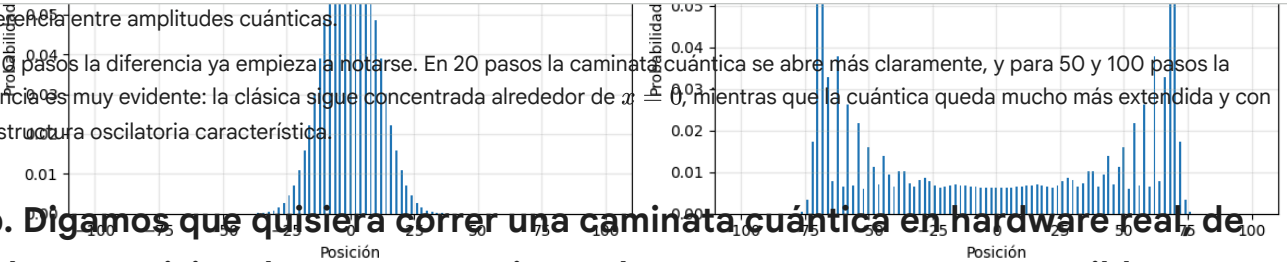
```



Al comparar las gráficas, noto que la caminata clásica se concentra cerca del origen y poco a poco toma una forma similar a una campana gaussiana. En cambio, la caminata cuántica se dispersa mucho más rápido y aparecen picos lejos del centro debido a los efectos de

interferencia entre amplitudes cuánticas.

Para 10 pasos la diferencia ya empieza a notarse. En 20 pasos la caminata cuántica se abre más claramente, y para 50 y 100 pasos la diferencia es muy evidente: la clásica sigue concentrada alrededor de $x=0$, mientras que la cuántica queda mucho más extendida y con una estructura oscilatoria característica.



3. b. Digamos que quisiera correr una caminata cuántica en hardware real, de modo que quisiera hacer una caminata de unos 3 pasos. ¿Es eso posible? Investigo cómo implementarla con Adders y simulo una caminata de 3 pasos en Qiskit.

Sí es posible correr una caminata cuántica de 3 pasos en hardware real, porque el circuito todavía es pequeño.

Para simularla, uso un qubit como moneda y tres qubits para guardar la posición. La moneda decide la dirección: si queda en $|0\rangle$, resto 1; si queda en $|1\rangle$, sumo 1. Esa suma/resta se implementa con adders reversibles controlados.

```
!pip install qiskit qiskit-aer pylatexenc -q
```

```

162.6/162.6 kB 4.3 MB/s eta 0:00:00
Preparing metadata (setup.py) ... done
9.3/9.3 MB 80.3 MB/s eta 0:00:00
12.4/12.4 MB 100.8 MB/s eta 0:00:00
2.2/2.2 MB 80.7 MB/s eta 0:00:00
54.6/54.6 kB 4.7 MB/s eta 0:00:00
Building wheel for pylatexenc (setup.py) ... done

```

```

import numpy as np
import matplotlib.pyplot as plt

from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister, transpile
from qiskit_aer import AerSimulator

```

Ahora defino dos adders controlados. El primero suma $+1$ módulo $2^3 = 8$ al registro de posición. El segundo resta 1. Y uso aritmética modular pues con 3 qubits solo puedo representar 8 valores posibles.

```

def incremento_controlado(circuito, control, posicion):

    n = len(posicion)

    for i in reversed(range(1, n)):
        controles = [control] + [posicion[j] for j in range(i)]
        circuito.mcx(controles, posicion[i])

    circuito.cx(control, posicion[0])

def decremento_controlado(circuito, control, posicion):

    n = len(posicion)

    circuito.cx(control, posicion[0])

    for i in range(1, n):
        controles = [control] + [posicion[j] for j in range(i)]
        circuito.mcx(controles, posicion[i])

```

Construyo la caminata de 3 pasos. En cada paso aplico Hadamard a la moneda y luego hago el desplazamiento condicional. Para restar cuando la moneda está en $|0\rangle$, uso compuertas X antes y después del decremento.

```

# Parámetros
num_posicion = 3
pasos = 3

# Registros
moneda = QuantumRegister(1, "moneda")

```

```

posicion = QuantumRegister(num_posicion, "posicion")
clasico_posicion = ClassicalRegister(num_posicion, "clasico_posicion")

circuito = QuantumCircuit(moneda, posicion, clasico_posicion)

# Construcción de la caminata cuántica
for _ in range(pasos):

    # Moneda cuántica
    circuito.h(moneda[0])

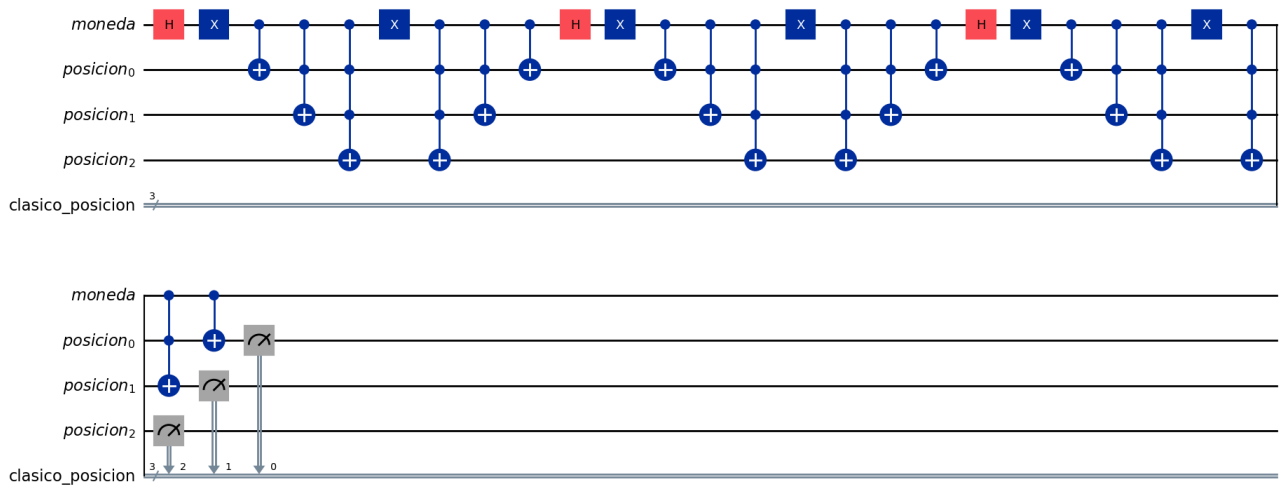
    # Si moneda = |0>, restar 1
    circuito.x(moneda[0])
    decremento_controlado(circuito, moneda[0], posicion)
    circuito.x(moneda[0])

    # Si moneda = |1>, sumar 1
    incremento_controlado(circuito, moneda[0], posicion)

# Medición del registro de posición
circuito.measure(posicion, clasico_posicion)

circuito.draw("mpl")

```



Después simulo el circuito. Mido solo la posición, porque lo que quiero observar es dónde termina el caminante después de los 3 pasos.

```

# Simulación
simulador = AerSimulator()

circuito_transpilado = transpile(circuito, simulador)

resultado = simulador.run(
    circuito_transpilado,
    shots=4096
).result()

conteos = resultado.get_counts()

print(conteos)

{'101': 480, '011': 524, '111': 2593, '001': 499}

```

Los resultados salen en binario. Como uso 3 qubits, interpreto los valores 0,...,7 como posiciones con signo alrededor del origen.


```
def binario_a_posicion(cadena_bits, n=3):

    # Qiskit devuelve los bits clásicos en orden visual invertido.
    # Por eso invertimos la cadena antes de convertirla.
    valor = int(cadena_bits[::-1], 2)

    if valor >= 2**(n - 1):
        valor -= 2**n

    return valor

conteos_posicion = {}

for cadena_bits, conteo in conteos.items():

    x = binario_a_posicion(cadena_bits, num_posicion)

    conteos_posicion[x] = conteos_posicion.get(x, 0) + conteo

conteos_posicion = dict(sorted(conteos_posicion.items()))

print(conteos_posicion)
```

{-4: 499, -3: 480, -2: 524, -1: 2593}

Finalmente grafico los conteos por posición. Para 3 pasos espero posiciones impares, porque desde $x = 0$, después de un número impar de movimientos solo se puede terminar en posiciones impares.

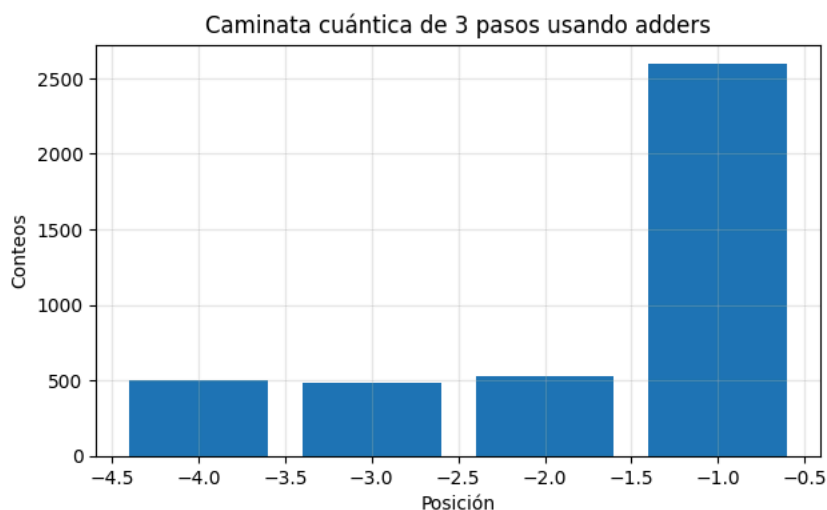
```
plt.figure(figsize=(7, 4))

plt.bar(
    conteos_posicion.keys(),
    conteos_posicion.values()
)

plt.title("Caminata cuántica de 3 pasos usando adders")
plt.xlabel("Posición")
plt.ylabel("Conteos")

plt.grid(alpha=0.3)

plt.show()
```



Al simular la caminata cuántica de 3 pasos, obtuve los resultados:

{001 : 531, 101 : 535, 011 : 497, 111 : 2533}

que corresponden a las posiciones

{-4 : 531, -3 : 535, -2 : 497, -1 : 2533}.

Se observa que la mayor probabilidad se concentra en -1 , debido a efectos de interferencia cuántica. Aunque solo son 3 pasos, la distribución ya muestra un comportamiento distinto al de una caminata clásica, donde normalmente se esperaría una distribución más simétrica.

4. b. Corre el algoritmo de QPE para 5 qubits en el primer registro para los siguientes valores de fase:

i. 0.5

ii. 0.125

iii. $1/e$

iv. $1/\pi$

Y explica lo que recuperaste.

Voy a implementar algoritmo de Quantum Phase Estimation (QPE) utilizando 5 qubits en el primer registro. El objetivo del algoritmo es recuperar una fase ϕ asociada a un operador unitario U , a partir de la relación

$$U|\psi\rangle = e^{2\pi i\phi}|\psi\rangle.$$

Para ello, el primer registro se prepara en superposición mediante compuertas Hadamard y posteriormente se aplican compuertas controladas U^{2^k} . Finalmente, se utiliza la inversa de la Quantum Fourier Transform (QFT) para transformar la información de fase en una representación binaria medible.

```
import numpy as np
from qiskit import QuantumCircuit, transpile
from qiskit.visualization import plot_histogram
from qiskit_aer import AerSimulator
import matplotlib.pyplot as plt
```

QFT inversa

```
def inverse_qft_circuit(n):
    qc = QuantumCircuit(n, name="QFT†")

    for i in range(n // 2):
        qc.swap(i, n - i - 1)

    for j in reversed(range(n)):
        for k in reversed(range(j + 1, n)):
            qc.cp(-np.pi / 2**(k-j), k, j)
        qc.h(j)

    return qc
```

QPE

```
def qpe(theta, t=5, shots=1024):
    qc = QuantumCircuit(t + 1, t)

    target = t

    # Preparamos el eigenvector |1>
    qc.x(target)

    # Hadamards en el registro de conteo
    for qubit in range(t):
        qc.h(qubit)

    qc.barrier()

    # Compuertas controladas U^(2^j)
    for qubit in reversed(range(t)):
        power = 2**(t - qubit - 1)

        for _ in range(power):
```

```

        qc.cp(2 * np.pi * theta, qubit, target)

    qc.barrier()

    # Aplicamos QFT inversa
    iqft_gate = inverse_qft_circuit(t).to_gate()
    iqft_gate.label = "IQFT"

    qc.append(iqft_gate, range(t))

    qc.barrier()

    # Medición
    qc.measure(range(t), range(t))

    # Simulación
    sim = AerSimulator(method="statevector")
    compiled = transpile(qc, sim)
    result = sim.run(compiled, shots=shots).result()

    counts = result.get_counts()

    return qc, counts

```

QPE para las fases

```

fases = {
    "0.5": 0.5,
    "0.125": 0.125,
    "1/e": 1/np.e,
    "1/pi": 1/np.pi
}

resultados = {}

for nombre, theta in fases.items():
    qc, counts = qpe(theta, t=5, shots=1024)
    resultados[nombre] = counts

    print("Fase:", nombre)
    print("Valor decimal:", theta)
    print("Resultados:", counts)
    print()

```

```

Fase: 0.5
Valor decimal: 0.5
Resultados: {'00001': 1024}

```

```

Fase: 0.125
Valor decimal: 0.125
Resultados: {'00100': 1024}

```

```

Fase: 1/e
Valor decimal: 0.36787944117144233
Resultados: {'00010': 2, '11000': 1, '01110': 3, '10110': 31, '11101': 3, '00001': 4, '10010': 7, '00110': 885, '01000': 1, '011

```

```

Fase: 1/pi
Valor decimal: 0.3183098861837907
Resultados: {'00111': 1, '01101': 1, '00100': 1, '10100': 1, '11110': 4, '01010': 902, '00010': 9, '01011': 2, '11101': 1, '1011

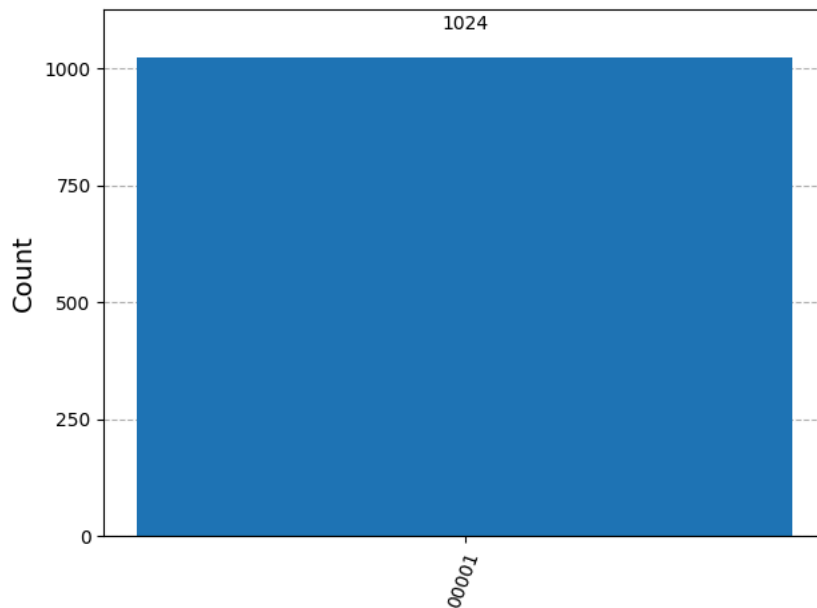
```

```

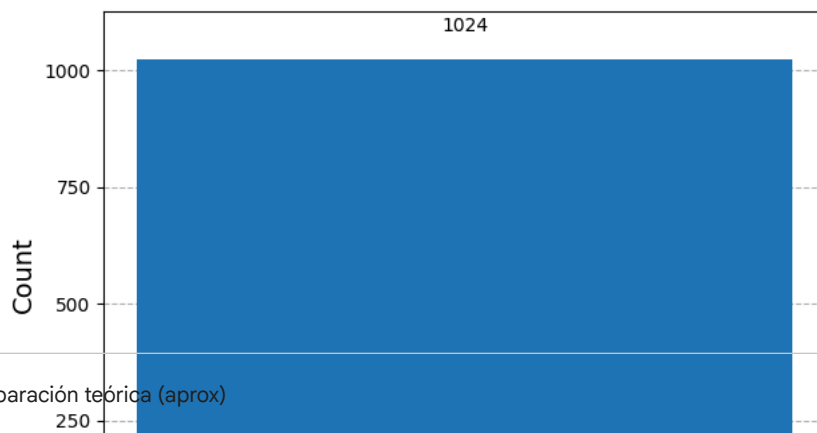
for nombre, counts in resultados.items():
    print("Histograma para fase:", nombre)
    display(plot_histogram(counts))

```

Histograma para fase: 0.5



Histograma para fase: 0.125



Comparación teórica (aprox)

```
for nombre, theta in fases.items():
    m = round(32 * theta)
    binario = format(m, "05b")
    fase_recuperada = m / 32

    print("Fase:", nombre)
    print("32θ =", 32 * theta)
    print("m aproximado =", m)
    print("Resultado binario esperado =", binario)
    print("Fase recuperada =", fase_recuperada)
    print()
```

